PROJECT REPORT

# Musical Instrument Recognition Using Digital Signal Processing and Machine Learning

Stage 1: Predominant Instrument Classification
Stage 2: Multi-Instrument Recognition

**Course: DIGITAL SIGNAL PROCESSING (ECN15101)**

**Submitted by**

Mohak Gupta • 20244106

Mohit Bagri • 20244107

Moksh Jain • 20244108

Mudit Agarwal • 20244109

**Group 4**




Department of Electronics and Communication Engineering
Motilal Nehru National Institute of Technology Allahabad

# Declaration

We hereby declare that the project report entitled **"Musical Instrument Recognition Using Digital Signal Processing and Machine Learning"** has been prepared by us as part of the requirements for the course **Digital Signal Processing (ECN15101)** at Motilal Nehru National Institute of Technology Allahabad.

This report presents our study and implementation of a two-stage musical instrument recognition system using digital signal processing techniques and convolutional neural networks. The work has been documented to the best of our knowledge, and all external sources, research papers, and references used in preparing this report have been appropriately acknowledged. We further declare that the work presented in this report reflects our collective efforts and understanding of the concepts studied during the course. We take responsibility for the accuracy and originality of the content presented and confirm that this report has been prepared for academic purposes.

**Group 4 — Section B1**

- Mohak Gupta (20244106)
- Mohit Bagri (20244107)
- Moksh Jain (20244108)
- Mudit Agarwal (20244109)

# Acknowledgment

We would like to express our sincere gratitude to our course instructor for providing us with the opportunity to undertake this project on musical instrument recognition using digital signal processing and machine learning. The project helped us develop a deeper understanding of audio signal processing, time-frequency analysis, Mel-spectrograms, convolutional neural networks, and performance evaluation techniques.

We are thankful to the Department of Electronics and Communication Engineering at Motilal Nehru National Institute of Technology Allahabad for providing the academic environment and resources necessary for completing this work. We also acknowledge the contributions of the researchers whose published work and the IRMAS dataset supported our study and implementation.

This project also provided us with an opportunity to strengthen our problem-solving abilities and apply theoretical concepts to a practical application. Through this work, we gained valuable experience in analysing audio data, designing learning models, and interpreting experimental results. We hope that this project will serve as a useful foundation for further exploration in digital signal processing and machine learning.

The knowledge and skills gained during this project have encouraged us to explore further applications of machine learning in audio analysis. We are grateful for the learning experience and the opportunity to work collaboratively on this technical project. Finally, we thank all our group members for their cooperation, discussions, and collective efforts in developing the system and preparing this report.

# Abstract

Automatic musical instrument recognition is an important task in music information retrieval, with applications in audio indexing, music organization, content-based retrieval, and automated musical analysis. Identifying instruments from audio recordings helps analyze musical arrangements and organize large audio collections. However, recognition remains challenging because instruments exhibit overlapping acoustic characteristics, and their spectral properties vary with pitch, playing technique, recording conditions, and the presence of other sound sources. These challenges are particularly significant in polyphonic recordings, where multiple instruments produce sound simultaneously. This project develops a two-stage musical instrument recognition system using digital signal processing (DSP) techniques and convolutional neural networks (CNNs) to perform predominant-instrument classification and multi-instrument recognition.

The proposed system uses the IRMAS (Instrument Recognition in Musical Audio Signals) dataset as its primary source of labelled recordings. Audio signals are processed at a sampling frequency of 22,050 Hz. Short-time Fourier transform (STFT) analysis is used to examine time-varying frequency content with a 2,048-point transform, a hop length of 512 samples, and a Hann window. The resulting spectral information is mapped onto 128 Mel frequency bands to generate Mel-spectrograms, which capture the temporal and spectral characteristics of musical sounds. These representations serve as input features for the CNN models, enabling them to learn discriminative patterns associated with different instrument classes.

The first stage focuses on predominant-instrument recognition as an 11-class single-label classification task. A CNN learns hierarchical features from Mel-spectrograms and assigns each recording to its predominant instrument class. Evaluated on held-out test data, the model achieves an accuracy of 75.25% and a test loss of 0.7513 on 1,006 samples. These results demonstrate the effectiveness of spectrogram-based feature extraction and CNN learning for predominant-instrument identification, although single-label classification is limited when several instruments contribute significantly to a recording.

The second stage extends the system to multilabel recognition, allowing multiple instrument classes to be predicted simultaneously. A separate multilabel CNN is trained on synthetically generated mixtures of two instruments, with balanced representation of instrument pairs. The model uses independent class probabilities and is evaluated on the official IRMAS test set containing 2,874 recordings. At a global decision threshold of 0.25, it achieves a micro-precision of 0.6743, a micro-recall of 0.5751, a micro-F1 score of 0.6208, and a macro-F1 score of 0.5317. Optimizing class-specific thresholds further improves the micro-F1 and macro-F1 scores to 0.6302 and 0.5344, respectively.

Overall, the project demonstrates a progression from predominant-instrument classification to multilabel recognition through DSP-based feature extraction, CNN architectures, synthetic-mixture training, and systematic evaluation. The results highlight the challenges of recognizing multiple instruments in real recordings, where overlapping timbres and recording variations can affect prediction accuracy. The developed framework provides a foundation for future improvements through more representative training data, enhanced feature extraction, and improved multilabel learning strategies. The comparison of both stages also illustrates the importance of selecting an appropriate classification strategy according to the complexity of the musical audio and the number of instruments present.

# Contents


| Contents | Page No. |
|---|---|
| **Individual Contributions of Group Members** | 5 |
| **1. Introduction** | 6 |
| **2. Related Work and Base Paper** | 6 |
| **3. Theoretical Background** | 7 |
| **3.1 Short-Time Fourier Transform (STFT)** | 7 |
| **3.2 Mel Spectrogram and dB Scale** | 8 |
| **3.3 CNN Building Blocks** | 8 |
| **3.4 Softmax versus Sigmoid** | 9 |
| **3.5 Metrics** | 9 |
| **4. Dataset and Signal Processing Pipeline** | 10 |
| **4.1 Dataset and Labels** | 10 |
| **4.2 Data Partitions** | 10 |
| **4.3 DSP Configuration** | 10–11 |
| **4.4 Workflow** | 11–12 |
| **5. Stage 1 — Predominant Instrument Recognition** | 13 |
| **5.1 Task Formulation** | 13 |
| **5.2 Training Configuration** | 13 |
| **5.3 Deeper CNN Architecture** | 14 |
| **5.4 Held-Out Results** | 15–16 |
| **5.5 Class-Wise Results** | 16 |
| **5.6 Confusion Matrix and Demonstration** | 17 |
| **6. Stage 2 — Multi-Instrument Recognition** | 18 |
| **6.1 Base-Paper Comparison and Work Beyond the Paper** | 18 |
| **6.2 Stage 2 — Multi-Instrument Recognition** | 18-23 |
| **7. Results and Discussion** | 24-25 |
| **8. Limitations and Future Work** | 25 |
| **9. Conclusion** | 26 |
| **References** | 26 |
| **Appendix A — Selected Implementation Code** | 27–29 |


# List of Figures


Figure 1. Audio signal frames used for short-time analysis.

Figure 2. Effect of the Hann window on an audio frame.

Figure 3. Short-time Fourier transform (STFT) spectrogram of the input audio.

Figure 4. Mel spectrogram of the input audio.

Figure 5. Overall workflow for predominant-instrument and multi-instrument recognition.

Figure 6. Mel spectrogram of an audio mixture used for multilabel recognition.

Figure 7. Waveform of the input audio signal.

Figure 8. Stage 1 CNN training and validation accuracy across epochs.

Figure 9. Stage 1 CNN training and validation loss across epochs.

Figure 10. Stage 1 per-class F1 scores on the held-out test set.

Figure 11. Confusion matrix for the deeper CNN instrument classifier.

Figure 12. Validation micro-F1 and macro-F1 across decision thresholds.

Figure 13. Stage 2 evaluation results on the official IRMAS test recordings.

Figure 14. Official IRMAS test performance using global and class-specific thresholds.

Figure 15. Stage 2 micro-F1 versus the number of annotated instruments (polyphony level).

## List of Tables

Table 1. Metric definitions used in evaluation

Table 2. IRMAS instrument labels and training counts

Table 3. Dataset partitions and evaluation roles

Table 4. Digital signal processing feature parameters

Table 5. Training hyperparameters for the deeper Stage 1 CNN

Table 6. Stage 1 CNN architecture summary (deeper CNN)

Table 7. Deeper Stage 1 CNN parameter totals

Table 8. Stage 1 held-out evaluation metrics

Table 9. Stage 1 class-wise precision, recall, and F1

Table 10. Initial Stage 2 random-mixture experiment

Table 11. Balanced-mixture dataset construction

Table 12. Stage 2 multilabel CNN architecture summary

Table 13. Stage 2 model parameter totals

Table 14. Stage 2 validation threshold comparison

Table 15. Official IRMAS test metrics by threshold strategy

Table 16. Class-specific thresholds and validation F1

Table 17. Official test performance by polyphony level

Table 18. Summary of key findings

Table 19. Base-paper comparison and work beyond the paper

Table 20. Class-wise performance on the official IRMAS test set

Table 21. Selected implementation code excerpts

# Individual Contributions of Group Members

## Mohak Gupta (20244106) — System Architecture, DSP Pipeline and Model Integration

Contributed to the conceptualization and technical design of the overall musical instrument recognition system. Established the two-stage recognition framework, covering predominant-instrument classification and multilabel recognition of multiple instruments. Worked on integrating audio preprocessing, Mel-spectrogram feature extraction, CNN-based classification, and multi-instrument prediction into a unified processing pipeline. Focused on maintaining consistency between the signal-processing stages and the inputs required by the deep learning models. Took primary responsibility for coordinating overall system development, integrating the individual components, and consolidating the experimental findings into the final project report.

## Mohit Bagri (20244107) — Deep Learning Model Architecture

Focused on the CNN-based approach for musical instrument recognition, with particular attention to model architecture and its application to audio feature classification. Examined how convolutional layers learn local time-frequency patterns from Mel-spectrogram representations and how successive layers develop more discriminative features for distinguishing instrument classes. Contributed to understanding the roles of pooling, activation functions, dropout, and fully connected layers in the classification process. Also examined the differences between single-label and multilabel prediction approaches and their relevance to the two stages of the proposed recognition system.

## Moksh Jain (20244108) — Dataset Characterization and Data Preparation

Focused on the IRMAS dataset, including its structure, instrument categories, labelling scheme, and suitability for musical instrument recognition. Examined how labelled audio recordings support the training and evaluation of the single-label classification model. Also considered the requirements of multilabel recognition, where a recording may contain multiple instruments and therefore require multiple target labels. Reviewed the role of dataset organization, consistent audio preprocessing, and the generation of synthetic two-instrument mixtures in preparing suitable inputs for model training. Contributed to understanding the importance of representative training examples and correctly organized labels for reliable model evaluation.

## Mudit Agarwal (20244109) — Experimental Evaluation and Performance Analysis

Focused on interpreting the experimental results and assessing the performance of the developed recognition system using appropriate evaluation metrics. Examined classification accuracy and test loss for predominant-instrument recognition, along with precision, recall, and F1-scores for multilabel recognition. Reviewed the significance of confusion matrices and prediction errors in identifying instrument classes that are more difficult to distinguish. Also considered the effect of decision-threshold selection on multilabel predictions and the differences between micro-averaged and macro-averaged performance measures. Contributed to discussing the strengths and limitations of the system, interpreting the experimental findings, and presenting the results in a clear and structured manner.

### Summary of Team Contributions

The team combined system design, deep learning, dataset preparation, and performance analysis to develop and evaluate a two-stage musical instrument recognition framework. These complementary responsibilities supported the integration of the DSP pipeline with CNN-based models and the systematic evaluation of both predominant-instrument classification and multilabel recognition.

# 1. Introduction

## 1.1 Background

Musical instrument recognition is a music-information-retrieval task with applications in automatic tagging, music indexing, audio search, and analysis of musical arrangements. Instrument timbres vary with pitch, playing technique, dynamics, room acoustics, and recording conditions. In polyphonic music, multiple sources overlap, making the task harder than recognizing a single isolated instrument.

## 1.2 Problem statement

The project develops a DSP-and-ML pipeline that converts audio into a time-frequency representation and uses CNNs to infer instrument labels. The work is deliberately separated into (i) single-label predominant-instrument classification and (ii) multilabel recognition where several instruments may be present simultaneously.

## 1.3 Objectives

- Load and inspect labelled IRMAS audio.
- Convert audio to a consistent sampling rate and extract fixed-shape Mel-spectrogram features.
- Train an 11-class CNN for predominant-instrument classification.
- Construct synthetic mixtures with multi-hot instrument targets for multilabel learning.
- Evaluate Stage 1 with accuracy and class-wise precision, recall, and F1.
- Evaluate Stage 2 with micro/macro metrics and compare global with class-specific thresholds on official IRMAS TestingData.

## 1.4 Scope and contribution

The project implements an end-to-end workflow linking Mel-spectrogram features, CNN training, synthetic mixtures, threshold selection, and evaluation on official annotated recordings. The reported models use Mel-spectrogram features; MFCC, modified group delay (ModGD), tempograms, and explainability are considered possible directions for future work.

# 2. Related Work and Base Paper

The base paper by Sivalingam et al. [1], addresses 11-class predominant-instrument recognition using Mel-spectrogram features and a CNN. It reports a peak validation accuracy of 78.37%, The paper uses a 128 × 128 × 1 input, whereas this project implementation uses 128 × 130 × 1. Stage 1 follows the same broad task formulation, while Stage 2 extends the project to multilabel recognition using synthetic mixtures, threshold selection, window aggregation, and evaluation on annotated IRMAS recordings.

The base paper provides a reference for understanding CNN-based instrument classification and the use of time–frequency representations for audio analysis. However, the two stages of this project address different prediction requirements: Stage 1 identifies a single predominant instrument, whereas Stage 2 aims to detect multiple instruments that may occur simultaneously. This extension introduces additional challenges, including selecting suitable decision thresholds and handling overlapping acoustic characteristics between instrument classes. The base paper therefore serves as a point of comparison for Stage 1, while Stage 2 explores a broader recognition task. All model configurations and experimental results reported in this project are kept separate from those published by the original authors.

Based on this comparison, we focused on extending the original single-instrument classification approach rather than simply reproducing the base paper's results. This allowed us to explore how CNN-based recognition could be adapted to identify multiple instruments within the same musical recording.

# 3. Theoretical Background

## 3.1 Short-Time Fourier Transform

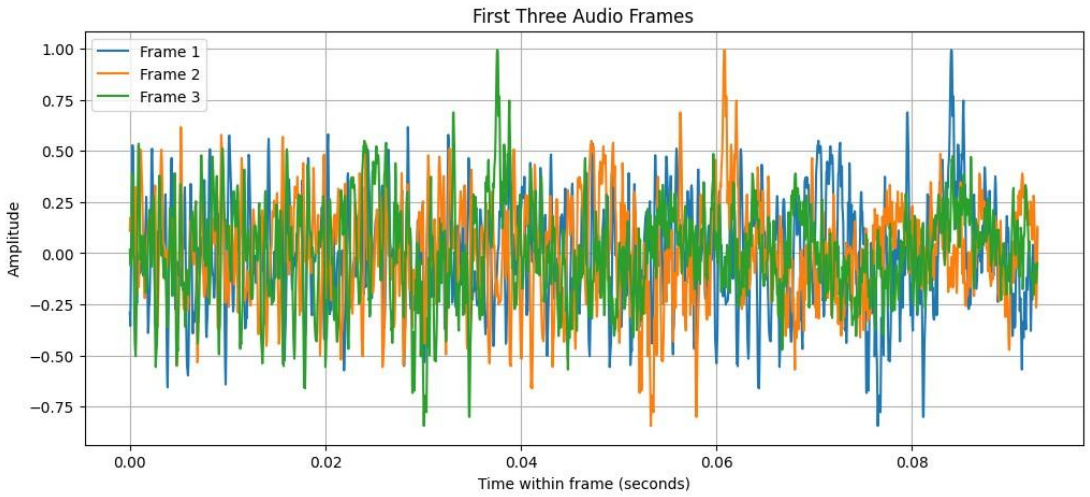



**Figure 1. Audio signal frames used for short-time analysis.**

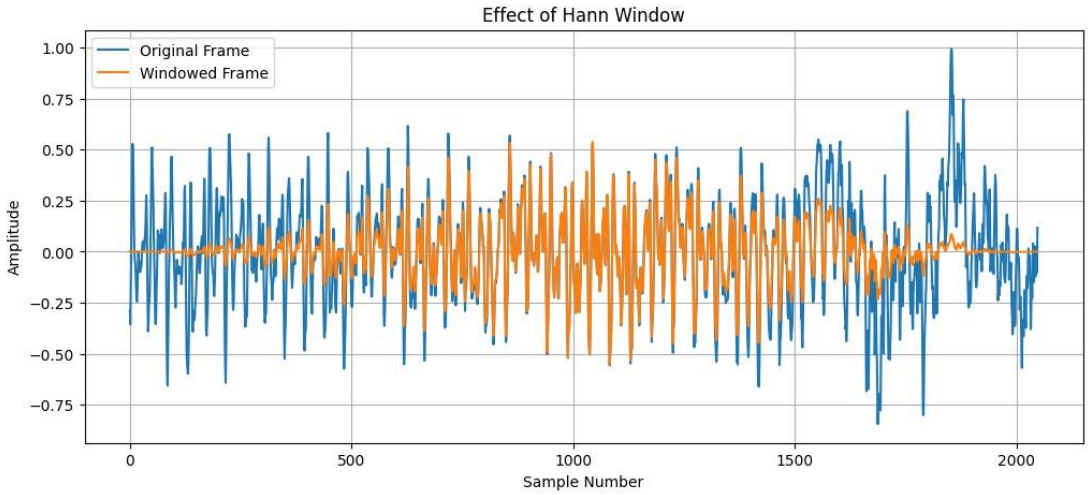



**Figure 2. Effect of the Hann window on an audio frame.**

Audio is non-stationary: spectral content changes over time. The short-time Fourier transform (STFT) computes a Fourier transform over overlapping, windowed frames:

$$X(m,k) = \Sigma[n=0\ldots N-1]\, x[n + mH]\, w[n]\, \exp(-j2\pi kn/N)$$

Here, N is the FFT size, H is the hop length, and w[n] is the analysis window. The implementation uses a Hann window, an FFT size of 2,048, and a hop length of 512.

The choice of STFT parameters affects the time–frequency resolution of the extracted representation. With an FFT size of 2,048 and a hop length of 512 samples, successive frames overlap substantially, allowing temporal continuity to be preserved while capturing important frequency details. This is particularly useful for musical signals, where harmonic structures and variations in spectral energy help distinguish instruments with similar timbral characteristics. A Hann window is applied to each frame to reduce spectral leakage caused by discontinuities at frame boundaries, improving the reliability of the resulting frequency representation. These design choices follow the Mel-spectrogram-based approach described in the base paper, which also uses a 2,048-point FFT and a 512-sample hop length. Furthermore, the resulting time–frequency representation provides a consistent basis for extracting instrument-specific patterns from audio recordings. This is important for the project because the quality of these features influences how effectively the CNN learns to distinguish between instrument classes during training.

## 3.2 Mel spectrogram and dB scale

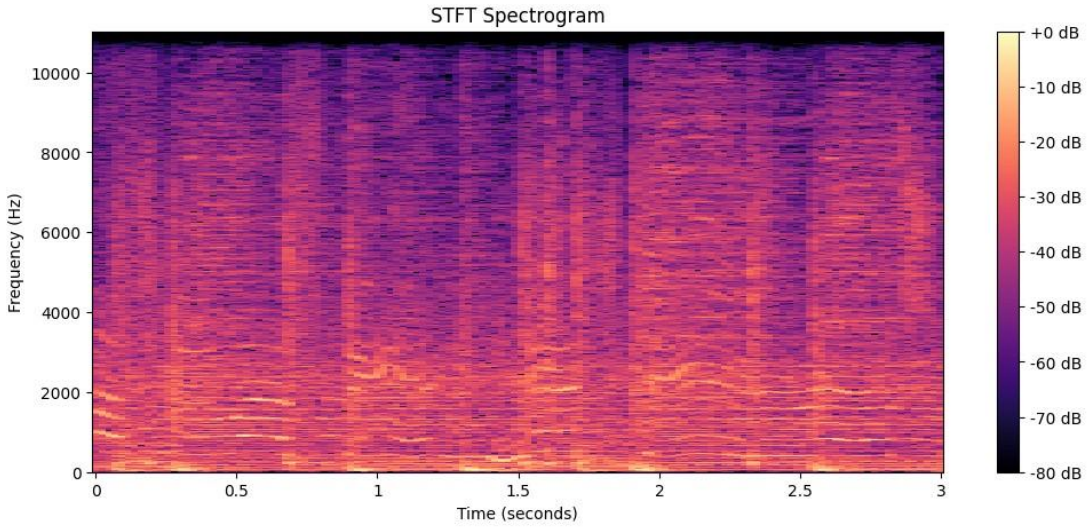



**Figure 3. Short-time Fourier transform (STFT) spectrogram of the input audio.**

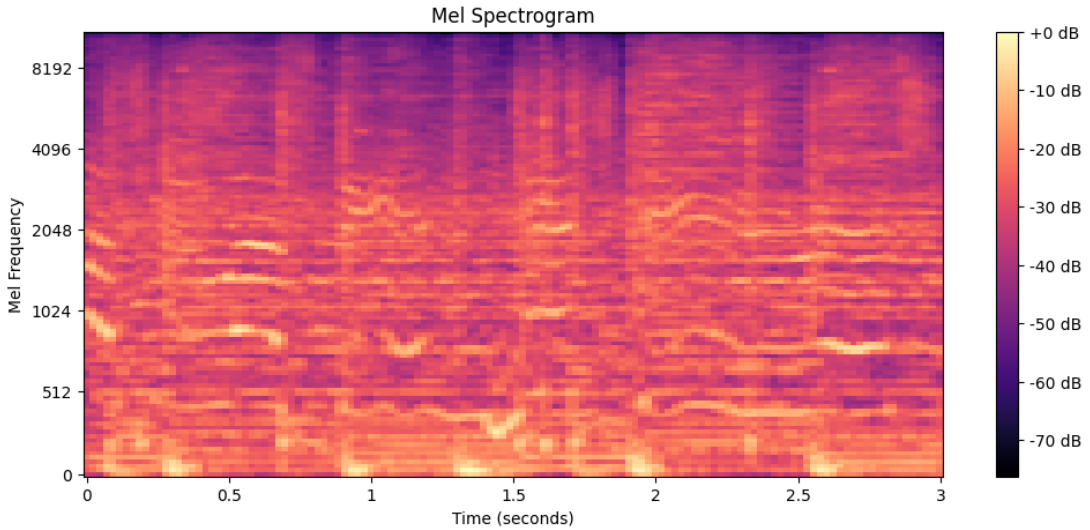



**Figure 4. Mel spectrogram of the input audio.**

The power spectrum is passed through a bank of Mel-spaced filters, reducing high-frequency resolution while retaining useful perceptual spectral structure. The implementation computes 128 Mel bands and converts the power spectrogram to decibels. The feature shape is 128 × 130; a channel axis gives the CNN input shape 128 × 130 × 1.

For Stage 2, the dB range from −80 to 0 dB is mapped into [0,1] using x_norm = (x + 80)/80. This fixed transformation is applied consistently to the feature representation.

## 3.3 CNN building blocks

Convolutional layers learn local time–frequency patterns from Mel-spectrograms, enabling the network to identify features associated with different musical instruments. Batch normalization stabilizes training by normalizing intermediate activations, while max pooling reduces spatial dimensions and computational requirements. Dropout randomly disables selected neurons during training to reduce overfitting and improve generalization to unseen audio samples. Global average pooling averages each final feature map into a single value, reducing the number of parameters compared with flattening. Dense layers combine the learned features to support the final classification decision. Together, these building blocks progressively learn complex representations, from basic spectral patterns to instrument-specific characteristics. The final output layer uses 11 units to represent the instrument classes, with softmax activation for predominant-instrument classification in Stage 1.

## 3.4 Softmax versus sigmoid

In multi-class classification, the choice of activation function depends on whether the target classes are mutually exclusive or can occur simultaneously. Softmax converts the output logits into a probability distribution in which the probabilities of all classes sum to one. In Stage 1 of this project, softmax is used because each audio clip is assigned a single predominant-instrument label among the 11 instrument classes. The class with the highest predicted probability is selected as the final prediction.

In contrast, sigmoid applies an independent probability function to each output neuron, allowing multiple classes to be predicted simultaneously. Stage 2 uses 11 independent sigmoid outputs because a musical recording may contain several instruments at the same time. Each output represents the predicted probability of a particular instrument being present, and a threshold determines whether that instrument is included in the final prediction. A global threshold applies the same decision boundary to all instrument classes, whereas class-specific thresholds allow different decision boundaries based on individual instrument characteristics. This distinction enables Stage 2 to extend the project beyond predominant-instrument classification towards multi-instrument recognition, where identifying all relevant instruments is more important than selecting only one class.

## 3.5 Metrics

| Metric | Definition | Interpretation |
|---|---|---|
| Precision | TP / (TP + FP) | Fraction of predicted positives that are correct. |
| Recall | TP / (TP + FN) | Fraction of true positives detected. |
| F1-score | 2PR / (P + R) | Harmonic mean of precision and recall. |
| Micro average | Aggregate TP, FP, FN, then compute | Weights classes according to total decisions. |
| Macro average | Compute each class metric, then average | Gives each class equal weight. |

**Precision, Recall and F1-score:**

Precision measures the proportion of predicted positive labels that are correct, indicating how reliably the model identifies musical instruments. Recall measures the proportion of actual positive labels correctly detected, showing the model's ability to identify instruments genuinely present in an audio recording. The F1-score is the harmonic mean of precision and recall, providing a balanced measure when both false-positive and false-negative predictions are important. These metrics help evaluate the instrument recognition system beyond accuracy, particularly when distinguishing between instruments with similar spectral characteristics.

**Micro Average:**

Micro averaging combines true positives, false positives, and false negatives across all instrument classes before calculating the metric. Consequently, classes contributing more decisions can have a greater influence on the final score. In this project, micro-averaged metrics help assess the overall effectiveness of the CNN in recognizing instruments across the dataset. They are particularly useful when the primary objective is to measure the combined performance of the recognition system.

**Macro Average:**

Macro averaging calculates the metric separately for each instrument class and then takes the unweighted average of these scores. Every instrument receives equal importance, regardless of its number of samples. In this project, macro-averaged metrics help determine whether the model performs consistently across all 11 instrument categories rather than performing well only on frequently represented classes. This is particularly useful for identifying weaknesses in the recognition of less-represented or acoustically similar instruments.

# 4. Dataset and Signal Processing Pipeline

## 4.1 Dataset and labels

The IRMAS dataset used in this project contains 6,705 labelled WAV files in IRMAS-TrainingData and 2,874 annotated WAV files in IRMAS-TestingData. Training clips are approximately three seconds long. The official test collection includes single-instrument and polyphonic recordings.

| Code | Instrument | Training files |
|---|---|---|
| cel | Cello | 388 |
| cla | Clarinet | 505 |
| flu | Flute | 451 |
| gac | Acoustic guitar | 637 |
| gel | Electric guitar | 760 |
| org | Organ | 682 |
| pia | Piano | 721 |
| sax | Saxophone | 626 |
| tru | Trumpet | 577 |
| vio | Violin | 580 |
| voi | Voice | 778 |
| Total | — | 6,705 |

## 4.2 Data partitions

| Partition | Samples | Share / role |
|---|---|---|
| Training | 4,693 | 70%; model fitting and training mixture source pool |
| Validation | 1,006 | 15%; model selection and threshold tuning |
| Internal test | 1,006 | 15%; held-out Stage 1 evaluation / internal multilabel evaluation where applicable |
| Official IRMAS TestingData | 2,874 tracks | Separate annotated collection; Stage 2 external-style evaluation |

## 4.3 DSP configuration

| Parameter | Implementation value | Purpose |
|---|---|---|
| Sampling rate | 22,050 Hz | Consistent audio sampling rate. |
| Window | Hann | Reduces spectral leakage at frame boundaries. |
| FFT size | 2,048 | Frequency resolution for each STFT frame. |
| Hop length | 512 samples | Frame step; overlapping frames. |

| Parameter | Implementation value | Purpose |
|---|---|---|
| Mel bands | 128 | Number of Mel filterbank channels. |
| Stored feature shape | 128 × 130 | Fixed time-frequency tensor. |
| CNN input | 128 × 130 × 1 | Single-channel CNN tensor. |
| Stage 2 dB range | −80 to 0 dB | Range observed before normalization. |
| Stage 2 normalization | (x + 80) / 80 | Maps [−80, 0] dB to [0, 1]. |

## 4.4 Workflow

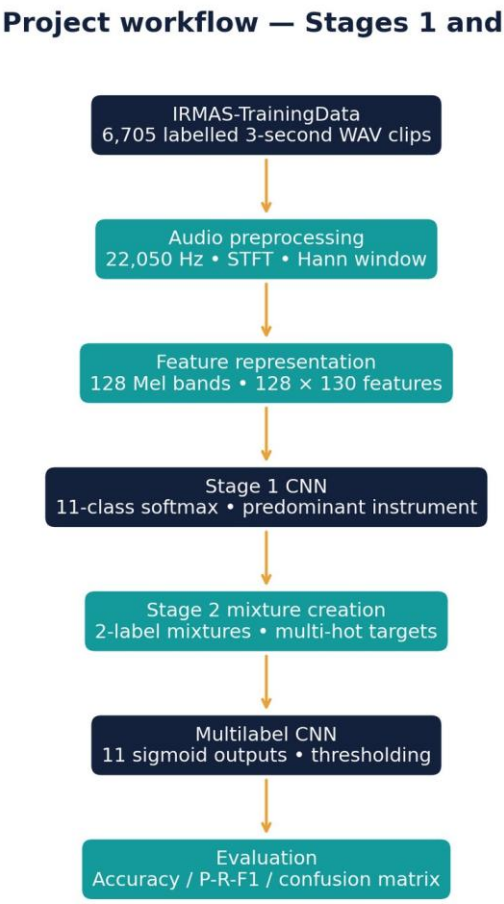



**Figure 5. Overall workflow for predominant-instrument and multi-instrument recognition.**

The overall workflow consists of audio preprocessing, feature extraction, and two stages of instrument recognition. First, the IRMAS TrainingData dataset, containing 6,705 labelled three-second WAV clips, is processed at a sampling rate of 22,050 Hz using the Short-Time Fourier Transform (STFT) and a Hann window. Mel-spectrograms with 128 Mel bands are then used as input features for the Stage 1 CNN, which performs 11-class predominant-instrument classification using a softmax output. **Stage 2 extends the work beyond the base paper** by addressing multi-instrument recognition rather than limiting the system to a single predominant instrument. It uses mixtures containing two or more instruments and trains a multi-label CNN with 11 sigmoid outputs to predict the presence of individual instrument classes. A thresholding mechanism converts the predicted probabilities into final instrument labels. Finally, the system is evaluated using accuracy, precision, recall, F1-score, and a confusion matrix, as appropriate for each stage.

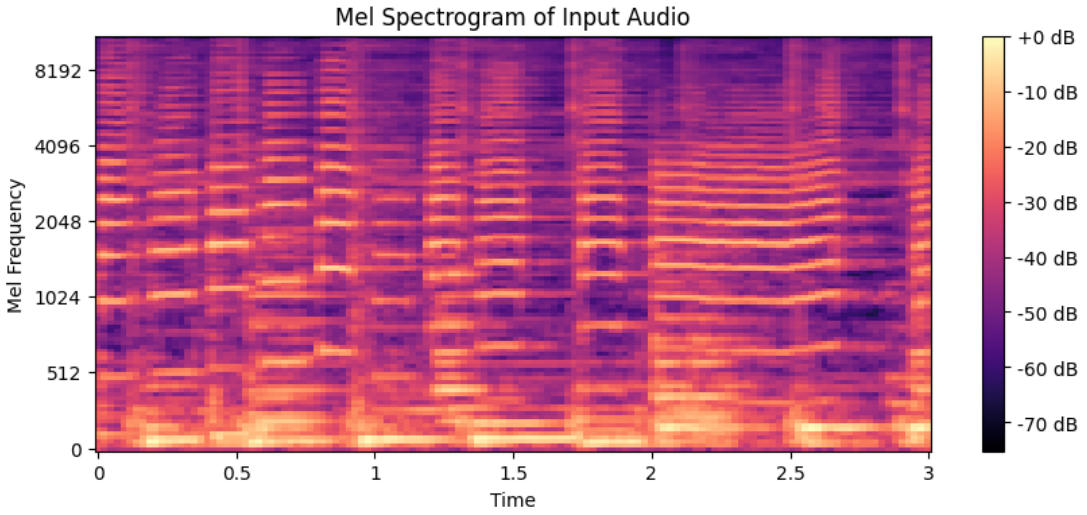



**Figure 6. Mel spectrogram of an audio mixture used for multilabel recognition.**

The pipeline converts each recording into a fixed-shape time-frequency representation. Stage 1 trains a single-label classifier. Stage 2 creates mixtures and multi-hot targets, trains a multilabel CNN, selects thresholds on validation data, and evaluates on official annotated recordings.

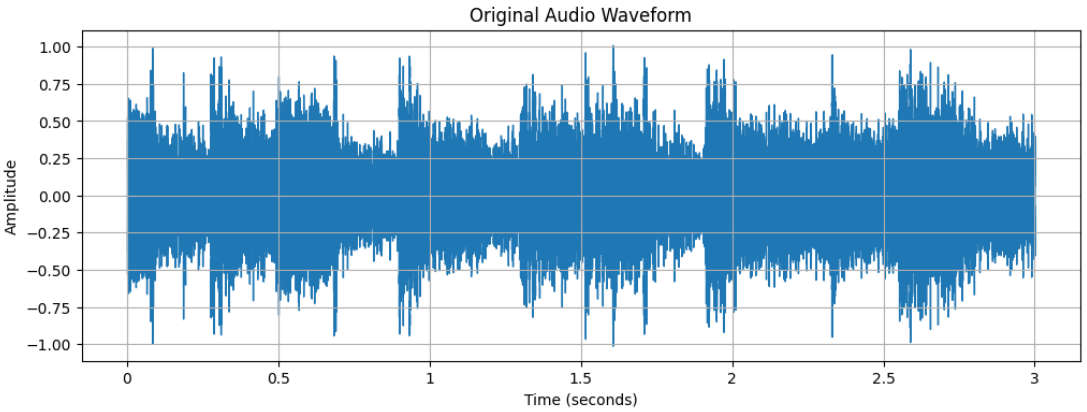



**Figure 7. Waveform of the input audio signal.**

The plotted time-frequency structure illustrates how spectral energy varies across time and Mel bands. Such structure is the image-like input consumed by the CNN; it is not itself a prediction.

**Interpretation of Audio Representations**

The waveform and Mel-spectrogram provide complementary views of the input audio signal. The waveform illustrates changes in amplitude over time, helping visualize variations in signal intensity and the temporal structure of the recording. In contrast, the Mel-spectrogram represents the distribution of energy across different frequency bands over time, making it easier to observe harmonic patterns and changes in spectral content. These representations are useful during preprocessing because they convert raw audio into a structured format suitable for deep learning.

The use of a consistent feature representation ensures that audio recordings can be processed uniformly before being supplied to the classification models. The CNN learns patterns from local time-frequency regions and combines them to identify features associated with different musical instruments. During training, the model adjusts its parameters to reduce prediction errors, while evaluation metrics are used to assess its ability to recognize instrument classes on unseen recordings. This process establishes a connection between signal-level analysis and automated musical instrument recognition.

# 5. Stage 1 — Predominant Instrument Recognition

## 5.1 Task formulation

Stage 1 is an 11-class single-label task. Each training clip has one predominant-instrument label. At inference, the class with the largest softmax probability is selected.

## 5.2 Training configuration

| Setting | Training configuration |
|---|---|
| Input tensor | 128 × 130 × 1 |
| Classes | 11 instrument labels |
| Initial learning rate | 0.001 |
| Batch size | 32 |
| Maximum epochs | 40 |
| L2 regularization coefficient | Not specified in the Stage 1 model definition |
| Dropout rate | 0.25 after each convolutional block; 0.50 after the dense layer |
| Optimizer | Adam |
| Loss function | Sparse categorical cross-entropy |
| Output layer | 11-unit softmax |
| Learning-rate reduction factor | 0.5 |
| Minimum learning rate | $1 \times 10^{-6}$ |
| Early stopping | Monitor: validation loss; patience: 7 epochs; restore best weights |
| ReduceLROnPlateau | Monitor: validation loss; patience: 3 epochs |
| Model checkpoint | Save best model based on validation loss |

The Stage 1 classifier uses a convolutional neural network (CNN) to classify each recording into one of 11 predominant-instrument classes.

The parameter choices for Stage 1 are based on the CNN-based approach described in the base paper, with adjustments to the training configuration used in our implementation. The initial learning rate of 0.001 and batch size of 32 support controlled model optimization, while the maximum of 40 epochs limits training duration. The Adam optimizer and sparse categorical cross-entropy loss are suitable for the 11-class, single-label classification task. Dropout rates of 0.25 after each convolutional block and 0.50 after the dense layer help reduce overfitting and improve generalization. A learning-rate reduction factor of 0.5, together with a minimum learning rate of \(1 \times 10^{-6}\), allows finer updates when validation loss stops improving. Early stopping with a patience of 7 epochs restores the best-performing weights, while ReduceLROnPlateau uses a patience of 3 epochs to adjust the learning rate. These settings combine efficient training with regularization and validation-based control, following the base paper's emphasis on adaptive learning-rate adjustment and overfitting prevention.

## 5.3 Deeper CNN architecture

Table 6 summarizes the deeper Stage 1 CNN. It uses two convolutional layers in each of three blocks, followed by global average pooling and a dense classification head. The model contains 306,155 parameters.

| Layer | Configuration | Activation | Output shape | Params |
|---|---|---|---|---|
| Input | Input tensor | — | 128 × 130 × 1 | 0 |
| Conv2D-1 | 32 filters; 3×3; same padding | ReLU | 128 × 130 × 32 | 320 |
| BatchNorm-1 | 32 channels | — | 128 × 130 × 32 | 128 |
| Conv2D-2 | 32 filters; 3×3; same padding | ReLU | 128 × 130 × 32 | 9,248 |
| BatchNorm-2 | 32 channels | — | 128 × 130 × 32 | 128 |
| MaxPool-1 | 2×2 | — | 64 × 65 × 32 | 0 |
| Dropout-1 | rate = 0.25 | — | 64 × 65 × 32 | 0 |
| Conv2D-3 | 64 filters; 3×3; same padding | ReLU | 64 × 65 × 64 | 18,496 |
| BatchNorm-3 | 64 channels | — | 64 × 65 × 64 | 256 |
| Conv2D-4 | 64 filters; 3×3; same padding | ReLU | 64 × 65 × 64 | 36,928 |
| BatchNorm-4 | 64 channels | — | 64 × 65 × 64 | 256 |
| MaxPool-2 | 2×2 | — | 32 × 32 × 64 | 0 |
| Dropout-2 | rate = 0.25 | — | 32 × 32 × 64 | 0 |
| Conv2D-5 | 128 filters; 3×3; same padding | ReLU | 32 × 32 × 128 | 73,856 |
| BatchNorm-5 | 128 channels | — | 32 × 32 × 128 | 512 |
| Conv2D-6 | 128 filters; 3×3; same padding | ReLU | 32 × 32 × 128 | 147,584 |
| BatchNorm-6 | 128 channels | — | 32 × 32 × 128 | 512 |
| MaxPool-3 | 2×2 | — | 16 × 16 × 128 | 0 |
| Dropout-3 | rate = 0.25 | — | 16 × 16 × 128 | 0 |
| GlobalAveragePooling2D | Spatial mean per channel | — | 128 | 0 |
| Dense | 128 units | ReLU | 128 | 16,512 |
| Dropout-4 | rate = 0.50 | — | 128 | 0 |
| Dense output | 11 units | Softmax | 11 | 1,419 |

| Parameter summary | Count |
|---|---|
| Total parameters | 306,155 |
| Trainable parameters | 305,259 |
| Non-trainable parameters | 896 |

## 5.4 Held-out results

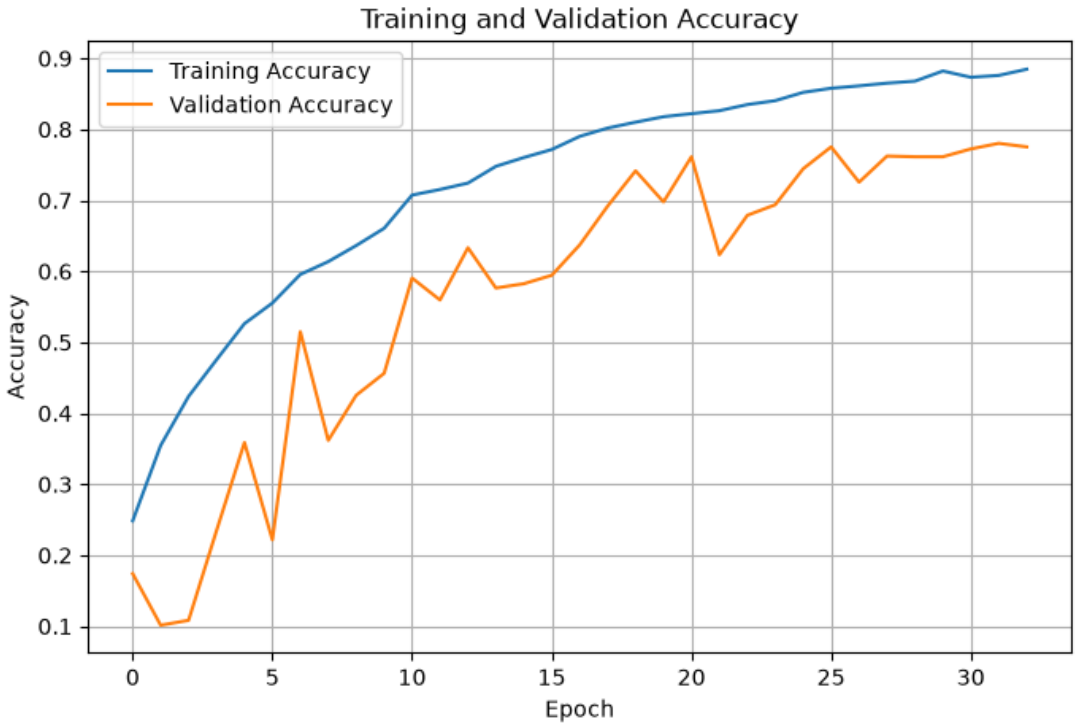



Figure 8. Stage 1 CNN training and validation accuracy across epochs.

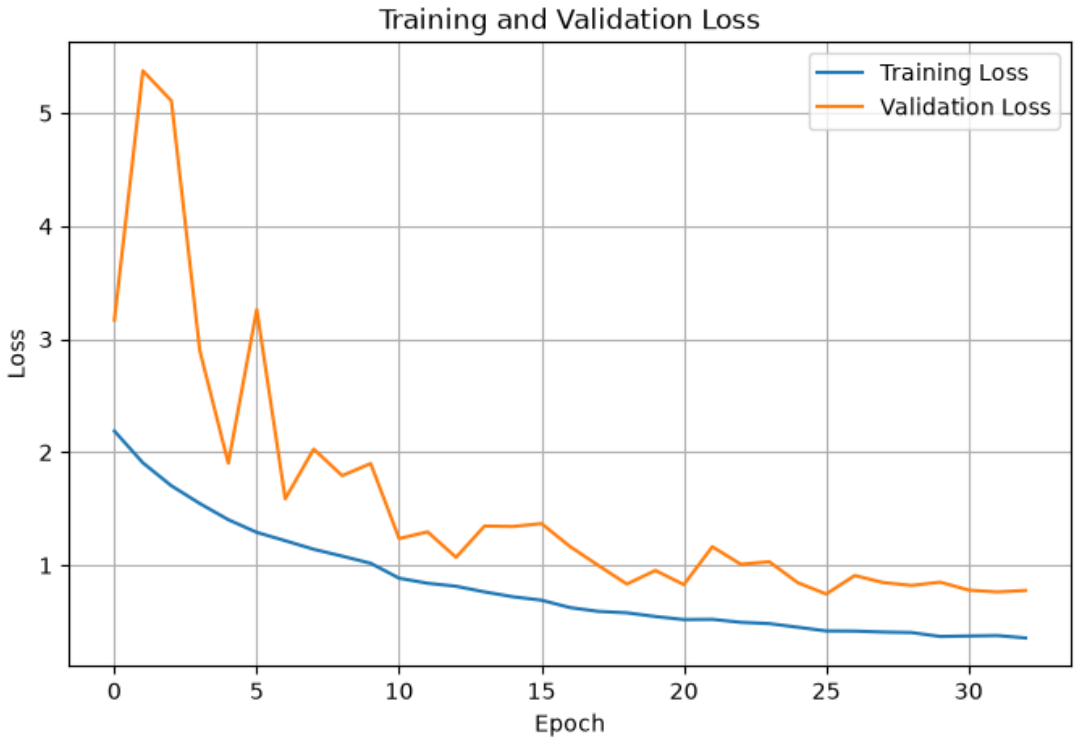



Figure 9. Stage 1 CNN training and validation loss across epochs.

| Metric | Saved result |
|---|---|
| Held-out sample count | 1,006 |
| Accuracy | 75.25% |
| Test loss | 0.7513 |
| Macro precision | 0.75 |

| Metric | Saved result |
|---|---|
| Macro recall | 0.73 |
| Macro F1 | 0.73 |
| Weighted F1 | 0.75 |

The 75.25% accuracy corresponds to roughly three out of four held-out recordings receiving the correct predominant label. Macro-F1 of 0.73 complements accuracy because it weights each class equally.

## 5.5 Class-wise results

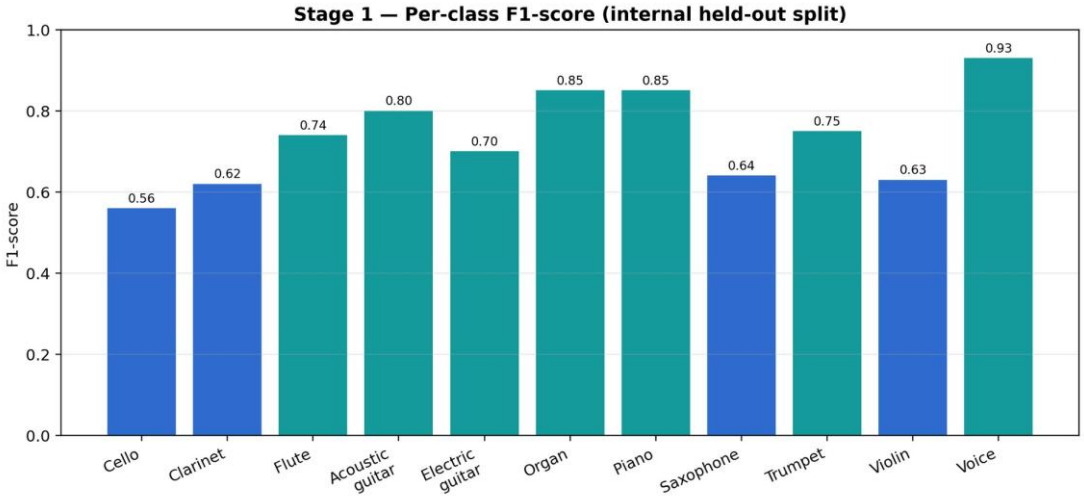



Figure 10. Stage 1 per-class F1 scores on the held-out test set.

| Instrument | Precision | Recall | F1 | Support |
|---|---|---|---|---|
| Cello | 0.69 | 0.47 | 0.56 | 58 |
| Clarinet | 0.83 | 0.50 | 0.62 | 76 |
| Flute | 0.77 | 0.72 | 0.74 | 68 |
| Acoustic guitar | 0.81 | 0.79 | 0.80 | 95 |
| Electric guitar | 0.63 | 0.81 | 0.70 | 114 |
| Organ | 0.83 | 0.88 | 0.85 | 103 |
| Piano | 0.86 | 0.84 | 0.85 | 108 |
| Saxophone | 0.68 | 0.60 | 0.64 | 94 |
| Trumpet | 0.69 | 0.83 | 0.75 | 86 |
| Violin | 0.59 | 0.69 | 0.63 | 87 |
| Voice | 0.94 | 0.91 | 0.93 | 117 |

Voice is strongest (F1 = 0.93), followed by organ and piano (approximately 0.85). Cello (0.56) and clarinet (0.62) are weaker. Cello recall of 0.47 indicates missed true cello clips; electric guitar has higher recall (0.81) but lower precision (0.63), consistent with more false-positive predictions.

## 5.6 Confusion matrix and demonstration

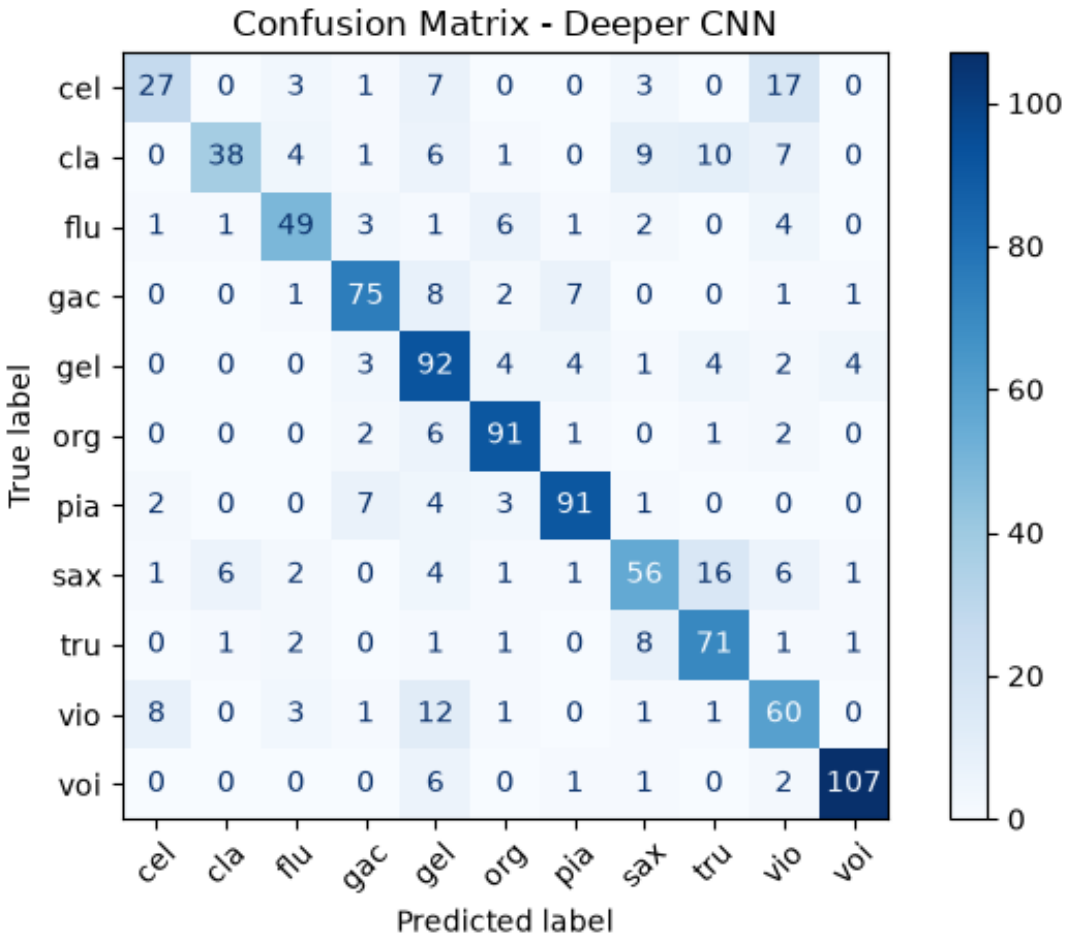



**Figure 11. Confusion matrix for the deeper CNN instrument classifier.**

The confusion matrix should be interpreted alongside per-class precision and recall: low recall indicates that true examples are assigned to other labels, while low precision means other instruments are incorrectly predicted as that class.

The confusion matrix provides a class-by-class view of the held-out predictions and complements the reported evaluation metrics. The diagonal elements of the confusion matrix represent correctly classified samples, whereas the off-diagonal elements indicate misclassifications between instrument classes. The model shows strong recognition of voice (107 correct predictions), electric guitar (92), organ (91), and piano (91), suggesting that it learns useful features for distinguishing these instruments.

In contrast, cello has 27 correct predictions, with 17 samples misclassified as violin, indicating difficulty distinguishing these two instruments. Similarly, 16 saxophone samples are classified as trumpet, while 12 violin samples are classified as electric guitar. Additionally, 10 clarinet samples are predicted as trumpet and 9 as saxophone. These errors may arise from overlapping harmonic structures and similar spectral characteristics in certain recordings.

Such patterns demonstrate why overall accuracy alone is insufficient to assess the classifier's performance. Per-class precision and recall help identify whether an instrument is frequently missed or incorrectly predicted. As discussed in the base paper, confusion between instruments with similar timbral characteristics remains a key challenge in CNN-based musical instrument recognition. Therefore, the confusion matrix provides useful evidence for identifying classes that may benefit from improved feature learning, data augmentation, or further model tuning.

A saved example for a 20-second recording reports trumpet as the predominant label with confidence around 73.57%. Window-level predictions can vary across time; this example demonstrates inference behavior and is not a substitute for held-out metrics.

# 6.1 Base-Paper Comparison and Work Beyond the Paper

Stage 1 follows the predominant-instrument CNN direction of Sivalingam et al. [1] and is the project component parallel to the base paper. Stage 2 is the extension developed in this project: multilabel outputs, synthetic multi-instrument mixtures, threshold selection, window aggregation, and evaluation on the official IRMAS test collection. The paper's values and our values come from different evaluation protocols and must not be interpreted as a controlled head-to-head experiment.

**Table 19. Comparison of Group 4 Stage 1 with Sivalingam et al. (2025)**

| Parameter / Evaluation Metric | Our Implementation — Stage 1 | Research Paper (Sivalingam et al., 2025) |
|---|---|---|
| Dataset | IRMAS | IRMAS |
| Audio duration and sampling rate | 3.0 s; 22,050 Hz | 3.0 s; 22,050 Hz |
| STFT (FFT / hop / Mel bands) | 2048 / 512 / 128 | 2048 / 512 / 128 |
| CNN trainable parameters | 306,155 | Approximately 440,075 |
| Data partitioning strategy | 70 : 15 : 15 (train : validation : test) | 80 : 20 (train : test) |
| Evaluation metric | Held-out test accuracy: 75.25% | Reported accuracy: approximately 78.47% (peak validation accuracy approximately 78.37%) |
| Classification output | 11-class softmax | 11-class softmax |

**Analytical comparison.** The Stage 1 implementation reaches 75.25% held-out test accuracy with 306,155 trainable parameters, compared with approximately 78.47% reported for the referenced paper and approximately 440,075 parameters. Thus, our implementation uses fewer trainable parameters, while its reported accuracy is lower by about 3.22 percentage points. This is not a strictly controlled head-to-head comparison: the data-partitioning strategies differ (70:15:15 versus 80:20), and the paper's reported validation metric should not be treated as equivalent to our held-out test accuracy. The shared dataset, audio duration, sampling rate, STFT settings, and 11-class softmax output provide useful points of methodological alignment.

# 6.2 Stage 2 — Multi-Instrument Recognition

## 6.2.1 Task formulation

A polyphonic recording may contain several instruments. Stage 2 therefore predicts an 11-dimensional multi-hot vector rather than one mutually exclusive class. A positive target indicates that an instrument is annotated as present.

## 6.2.2 Initial random-mixture experiment

| Item | Initial model record |
|---|---|
| Training mixtures | 4,693 synthetic two-instrument mixtures |
| Validation mixtures | 1,006 synthetic two-instrument mixtures |
| Target vector | 11 labels; exactly two positive labels per mixture |
| Output | 11 sigmoid probabilities |
| Selected threshold | 0.25 |
| Best saved validation F1 | 0.5413 |

The initial random-mixture model is an earlier experiment and must not be conflated with the improved balanced-mixture v2 model. Its validation threshold-sweep F1 is reported separately.

### 6.2.3 Balanced-mixture model (v2)

With 11 classes there are 55 unique unordered instrument pairs. The project uses 4,675 two-instrument mixtures (55 × 85), 3,300 three-instrument mixtures (165 triplets × 20), and 4,693 original single-instrument training examples. The combined training set contains 12,668 examples; validation includes 990 two-instrument and 990 three-instrument mixtures, for 1,980 examples.

| Dataset component | Construction | Samples | Positive labels/sample |
|---|---|---|---|
| Original single-instrument | 4,693 source clips | 4,693 | 1 |
| Balanced two-instrument | 55 pairs × 85 | 4,675 | 2 |
| Three-instrument training | 165 triplets × 20 | 3,300 | 3 |
| Two-instrument validation | 55 pairs × 18 | 990 | 2 |
| Three-instrument validation | 165 triplets × 6 | 990 | 3 |
| Combined v2 training | Single + pair + triplet | 12,668 | 1, 2, or 3 |
| Combined v2 validation | Pair + triplet | 1,980 | 2 or 3 |

Synthetic mixtures provide controllable labels and class combinations, but they do not reproduce every acoustic interaction found in real recordings. This distinction helps explain why official polyphonic performance is more challenging.

Motivation for balancing the pairs. The initial random-mixture experiment showed substantial variation in pair frequency: Piano–Voice appeared 149 times, Organ–Voice 136 times, and Electric Guitar–Saxophone 135 times, whereas Cello–Flute appeared 41 times and Cello–Trumpet and Cello–Acoustic Guitar appeared 44 times each. The most frequent example listed was therefore about 3.6 times as common as the least frequent example. This observed sampling skew motivated the balanced v2 construction, which assigns 85 training mixtures to each of the 55 unordered instrument pairs.

### 6.2.4 Stage 2 CNN architecture summary

The Stage 2 multilabel CNN uses convolutional blocks with same-padded 3×3 kernels and 32, 64, and 128 filters. Each block contains two convolutions, batch normalization, max pooling, and 0.25 dropout. Global average pooling feeds a 128-unit ReLU dense layer and 0.50 dropout, followed by 11 sigmoid outputs.

| Layer | Configuration | Activation | Output shape | Params |
|---|---|---|---|---|
| Input | Input tensor | — | 128 × 130 × 1 | 0 |
| Conv2D-1 | 32 filters; 3×3; same | ReLU | 128 × 130 × 32 | 320 |
| BatchNorm-1 | 32 channels | — | 128 × 130 × 32 | 128 |
| Conv2D-2 | 32 filters; 3×3; same | ReLU | 128 × 130 × 32 | 9,248 |
| BatchNorm-2 | 32 channels | — | 128 × 130 × 32 | 128 |
| MaxPool-1 | 2×2 | — | 64 × 65 × 32 | 0 |
| Dropout-1 | rate = 0.25 | — | 64 × 65 × 32 | 0 |
| Conv2D-3 | 64 filters; 3×3; same | ReLU | 64 × 65 × 64 | 18,496 |
| BatchNorm-3 | 64 channels | — | 64 × 65 × 64 | 256 |
| Conv2D-4 | 64 filters; 3×3; same | ReLU | 64 × 65 × 64 | 36,928 |

| Layer | Configuration | Activation | Output shape | Params |
|---|---|---|---|---|
| BatchNorm-4 | 64 channels | — | 64 × 65 × 64 | 256 |
| MaxPool-2 | 2×2 | — | 32 × 32 × 64 | 0 |
| Dropout-2 | rate = 0.25 | — | 32 × 32 × 64 | 0 |
| Conv2D-5 | 128 filters; 3×3; same | ReLU | 32 × 32 × 128 | 73,856 |
| BatchNorm-5 | 128 channels | — | 32 × 32 × 128 | 512 |
| Conv2D-6 | 128 filters; 3×3; same | ReLU | 32 × 32 × 128 | 147,584 |
| BatchNorm-6 | 128 channels | — | 32 × 32 × 128 | 512 |
| MaxPool-3 | 2×2 | — | 16 × 16 × 128 | 0 |
| Dropout-3 | rate = 0.25 | — | 16 × 16 × 128 | 0 |
| GlobalAveragePooling2D | Spatial mean per channel | — | 128 | 0 |
| Dense | 128 units | ReLU | 128 | 16,512 |
| Dropout | rate = 0.50 | — | 128 | 0 |
| Dense output | 11 units | Sigmoid | 11 | 1,419 |

| Parameter summary | Count |
|---|---|
| Total parameters | 306,155 |
| Trainable parameters | 305,259 |
| Non-trainable parameters | 896 |

Batch-normalization layers account for the non-trainable moving statistics. Dropout, pooling, and global average pooling have no learned parameters. The model compiles with Adam, binary cross-entropy, binary accuracy, and multilabel AUC. Callbacks use early stopping on validation loss (patience 7), ReduceLROnPlateau (factor 0.5, patience 3, minimum learning rate 1e−6), and best-only checkpointing.

### 6.2.5 Validation threshold selection

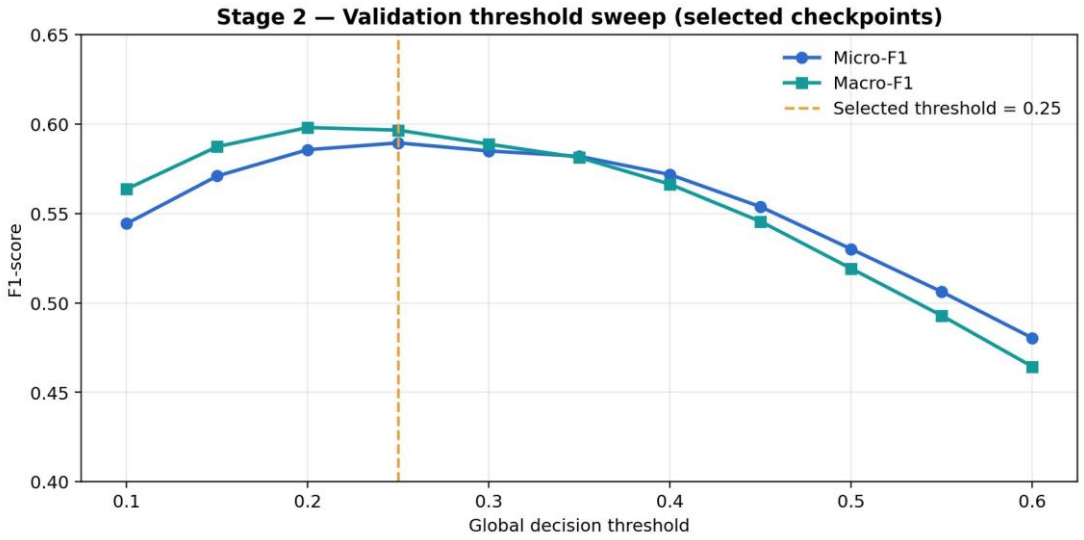



Figure 12. Validation micro-F1 and macro-F1 across decision thresholds.

The threshold sweep evaluates candidate thresholds starting at 0.10. The maximum validation micro-F1 occurs at threshold 0.25: micro-F1 = 0.5894 and macro-F1 = 0.5965. Macro-F1 at threshold 0.20 is slightly higher (0.5980), while 0.25 is selected to maximize micro-F1.

| Threshold | Validation micro-F1 | Validation macro-F1 |
|---|---|---|
| 0.10 | 0.5443 | 0.5636 |
| 0.15 | 0.5709 | 0.5873 |
| 0.20 | 0.5856 | 0.5980 |
| 0.25 | 0.5894 | 0.5965 |
| 0.30 | 0.5849 | 0.5887 |
| 0.35 | 0.5819 | 0.5812 |
| 0.40 | 0.5717 | 0.5663 |
| 0.50 | 0.5301 | 0.5192 |

Thresholding controls the balance between false positives and false negatives. Lower thresholds increase recall but can over-predict labels; higher thresholds generally reduce false positives while missing more present instruments.

### 6.2.6 Official IRMAS evaluation

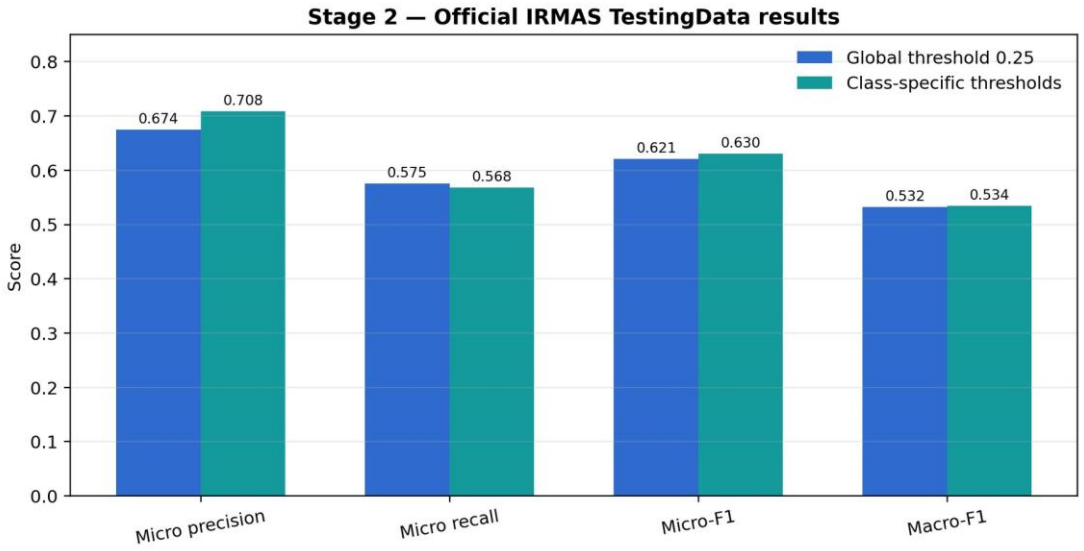



**Figure 13. Stage 2 evaluation results on the official IRMAS test recordings.**

The model is evaluated on 2,874 official WAV files, extracting 3-second windows with a 1.5-second hop and aggregating window probabilities. This differs from the controlled synthetic validation mixture setup and tests generalization to annotated real recordings.

| Metric | Global threshold 0.25 | Class-specific thresholds |
|---|---|---|
| Micro precision | 0.6743 | 0.7082 |
| Micro recall | 0.5751 | 0.5676 |
| Micro-F1 | 0.6208 | 0.6302 |
| Macro-F1 | 0.5317 | 0.5344 |

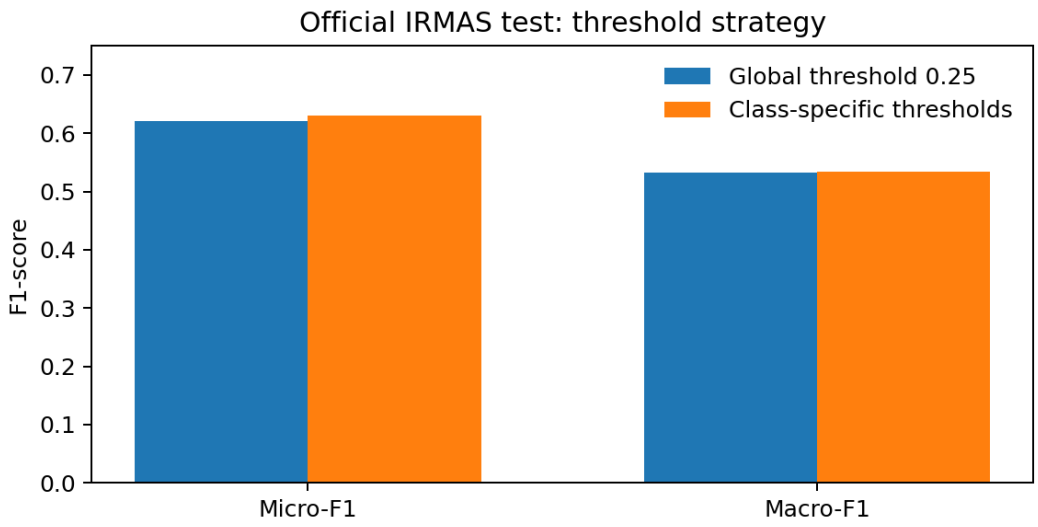



Figure 14. Official IRMAS test performance using global and class-specific thresholds.

Class-specific thresholds improve micro-F1 by about 0.0094 and macro-F1 by about 0.0027, with higher precision and slightly lower recall. Thresholds are tuned on validation data and then applied to the official set; they are not selected using official test labels.

### 6.2.7 Class-specific thresholds

| Class | Threshold | Validation F1 |
|---|---|---|
| Cello | 0.175 | 0.5402 |
| Clarinet | 0.350 | 0.5874 |
| Flute | 0.400 | 0.5580 |
| Acoustic guitar | 0.175 | 0.6067 |
| Electric guitar | 0.475 | 0.5346 |
| Organ | 0.450 | 0.6863 |
| Piano | 0.125 | 0.6955 |
| Saxophone | 0.225 | 0.5259 |
| Trumpet | 0.200 | 0.6896 |
| Violin | 0.175 | 0.5792 |
| Voice | 0.125 | 0.8169 |

Table 20. Class-wise performance on the official IRMAS test set

| Instrument | Threshold | Precision | Recall | F1-score |
|---|---|---|---|---|
| Cello | 0.175 | 0.218 | 0.387 | 0.279 |
| Clarinet | 0.350 | 0.250 | 0.290 | 0.269 |
| Flute | 0.400 | 0.431 | 0.558 | 0.487 |
| Acoustic guitar | 0.175 | 0.606 | 0.507 | 0.552 |
| Electric guitar | 0.475 | 0.791 | 0.483 | 0.584 |
| Organ | 0.450 | 0.577 | 0.291 | 0.387 |
| Piano | 0.125 | 0.845 | 0.406 | 0.549 |

| Instrument | Threshold | Precision | Recall | F1-score |
|---|---|---|---|---|
| Saxophone | 0.225 | 0.756 | 0.810 | 0.782 |
| Trumpet | 0.200 | 0.418 | 0.707 | 0.526 |
| Violin | 0.175 | 0.570 | 0.521 | 0.545 |
| Voice | 0.125 | 0.951 | 0.892 | 0.920 |

**Interpretation.** Voice has the highest class-wise F1 (0.920), followed by saxophone (0.782), while cello (0.279) and clarinet (0.269) are the weakest under the selected class-specific thresholds. The large variation across classes supports reporting per-class metrics alongside aggregate scores. These are official-test results at thresholds selected using validation data.

### 6.2.8 Performance versus polyphony

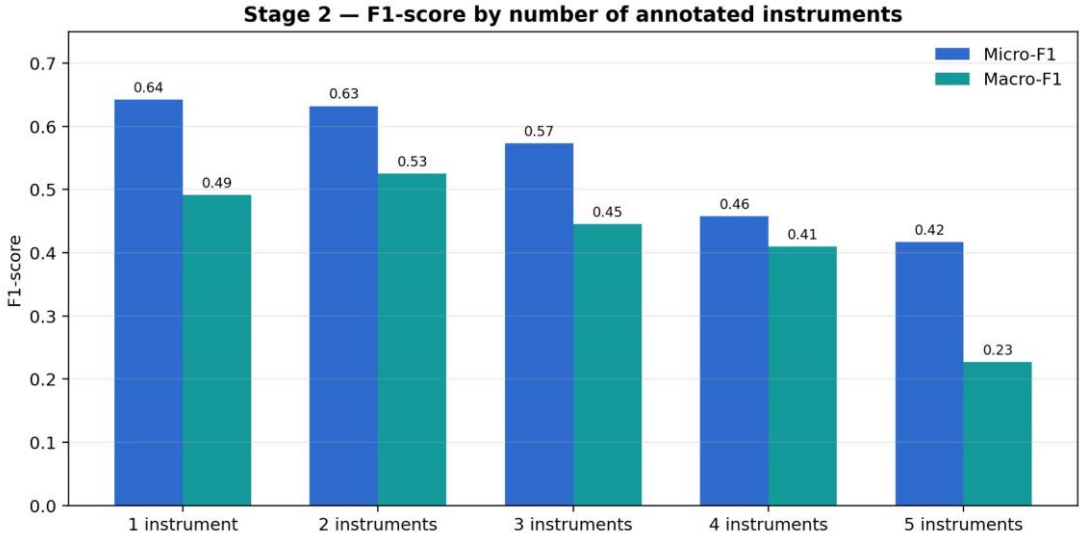



Figure 15. Stage 2 micro-F1 versus the number of annotated instruments (polyphony level).

| Annotated instruments | Files | Micro-F1 | Macro-F1 |
|---|---|---|---|
| 1 | 1,236 | 0.6423 | 0.4919 |
| 2 | 1,278 | 0.6322 | 0.5255 |
| 3 | 318 | 0.5735 | 0.4455 |
| 4 | 39 | 0.4581 | 0.4095 |
| 5 | 3 | 0.4167 | 0.2273 |

The results show a downward trend in micro-F1 as the number of annotated instruments increases, although the four- and five-instrument groups are small. These results suggest that overlapping sources and the limited number of high-polyphony recordings make recognition harder.

### 6.2.9 Long-audio inference case study

A 20-second recording, “02. School Boy-9.wav,” was processed using overlapping 3-second windows with a 1.5-second hop, yielding 13 windows. The supplied case-study result lists Piano and Trumpet as the ground-truth instruments. At the global threshold of 0.25, Trumpet was detected with an aggregated probability of 65.70%, while Acoustic Guitar received 26.67% and was a false positive. Piano was not reported among the detections shown in the presentation, illustrating that a correct prediction for one instrument does not guarantee recovery of every instrument in a polyphonic recording. This is an illustrative case, not a replacement for aggregate evaluation across the official test set.

# 7. Results and Discussion

## 7.1 Key findings

| Finding | Evidence | Interpretation |
|---|---|---|
| Stage 1 reaches useful single-label performance | 75.25% accuracy; 0.7513 loss | Predominant instrument is recoverable for many clips, but class-wise performance varies. |
| Voice is the strongest Stage 1 class | F1 = 0.93 | Its timbral patterns may be more separable in this dataset; this is an observation, not a causal proof. |
| Balanced mixture v2 improves multilabel validation | Micro-F1 = 0.5894 at $\tau = 0.25$ | Balanced pair/triplet construction expands the training target combinations. |
| Official test differs from synthetic validation | Official micro-F1 = 0.6208 at global $\tau = 0.25$ | Synthetic and real recordings differ in acoustic composition and annotation distribution. |
| Per-class thresholds improve official F1 modestly | Micro-F1 = 0.6302; macro-F1 = 0.5344 | Per-class calibration changes the precision–recall balance. |
| More polyphony is associated with lower F1 | Micro-F1 falls from 0.6423 (1 label) to 0.4167 (5 labels) | The highest-polyphony groups have few files, so estimates are less stable. |

## 7.2 Demonstration of Final Project Output

To demonstrate the performance of the proposed system, a 20-second audio recording containing piano and trumpet was provided as input. The system processed the recording using both recognition stages, allowing single-instrument and multi-instrument predictions to be compared with the actual instrument labels.

**Stage 1 — Predominant Instrument Recognition**

Stage 1 predicted *Trumpet* as the predominant instrument, with a confidence of 40.53%. Trumpet received the highest probability among the 11 instrument classes, followed by saxophone (24.93%) and clarinet (13.13%). Since trumpet was one of the instruments actually present in the recording, the prediction was considered correct. However, the relatively low confidence indicates some uncertainty in distinguishing the predominant instrument from acoustically similar classes.

**Stage 2 — Multi-Instrument Recognition**

Stage 2 predicted trumpet with a probability of 47.06%, exceeding its class-specific threshold of 0.200. However, piano received a probability of only 4.41%, below its threshold of 0.125, and was therefore not detected. Although trumpet was correctly identified, the system missed the piano component, resulting in a partially correct multi-instrument prediction. This illustrates the additional challenge of detecting every instrument in a recording rather than identifying only the predominant one.

**Window-Wise Prediction Analysis**

The recording was also evaluated using overlapping time windows to observe how predictions varied throughout the audio. Clarinet received high prediction scores in the early windows, while trumpet became more prominent in several subsequent windows. Saxophone was predicted in some intermediate windows, and piano reached a probability of 51.8% in the 13.5–16.5-second window. These variations demonstrate that the predicted instrument probabilities can change over time as the musical content and acoustic characteristics vary. However, window-level predictions should be interpreted cautiously because a high probability for a class does not necessarily confirm that the instrument is present.

```
==========================================
STAGE 1 — SINGLE-INSTRUMENT RECOGNITION
==========================================

Predicted Instrument: Trumpet
Confidence: 40.53%

Stage 1 Result: CORRECT

Stage 1 Class Probabilities:
Trumpet             40.53%
Saxophone           24.93%
Clarinet            13.13%
Piano               8.00%
Violin              6.77%
Flute               3.78%
Electric Guitar     1.44%
Acoustic Guitar     0.66%
Cello               0.62%
Organ               0.11%
Voice               0.01%
```

```
======================================================
STAGE 2 — MULTI-INSTRUMENT RECOGNITION
======================================================

Predicted Instruments:
Trumpet             47.06%  (threshold: 0.200)

Stage 2 Result: PARTIALLY CORRECT

Correctly Detected:
  ✓ Trumpet

Missed Instruments:
  X Piano

Stage 2 Class Probabilities:
Trumpet             47.06% (threshold: 0.200) DETECTED
Clarinet            33.48% (threshold: 0.350)
Saxophone           21.36% (threshold: 0.225)
Flute               11.43% (threshold: 0.400)
Piano               4.41% (threshold: 0.125)
Violin              1.28% (threshold: 0.175)
Cello               1.04% (threshold: 0.175)
Electric Guitar     0.81% (threshold: 0.475)
Acoustic Guitar     0.22% (threshold: 0.175)
Organ               0.11% (threshold: 0.450)
Voice               0.00% (threshold: 0.125)
```

```
======================================================================
WINDOW-WISE STAGE 2 PREDICTIONS
======================================================================
0.0-3.0 s : Clarinet (78.4%), Trumpet (31.0%)
1.5-4.5 s : Clarinet (87.8%)
3.0-6.0 s : Clarinet (60.8%), Trumpet (45.8%)
4.5-7.5 s : Trumpet (95.6%)
6.0-9.0 s : Trumpet (40.6%), Clarinet (39.1%), Saxophone (35.1%)
7.5-10.5 s : Saxophone (34.1%)
9.0-12.0 s : Saxophone (38.9%), Trumpet (35.9%)
10.5-13.5 s : Saxophone (63.5%)
12.0-15.0 s : Clarinet (40.6%), Saxophone (28.6%)
13.5-16.5 s : Piano (51.8%), Trumpet (32.6%)
15.0-18.0 s : Trumpet (99.6%)
16.5-19.5 s : Trumpet (99.8%)
18.0-20.0 s : Trumpet (85.0%)
```

# 8. Limitations and Future Work

• Stage 1 performance can be further improved through architecture tuning and more systematic model selection.

• Synthetic mixtures have controlled labels but may not fully represent real polyphonic mixing, masking, room acoustics, or instrument interactions.

• The official test has an uneven label distribution and relatively few examples with four or five annotated instruments; class- and polyphony-specific metrics should be interpreted with support counts.

• Threshold selection materially affects precision and recall. Future evaluation should preserve a held-out validation set for threshold tuning and avoid selecting thresholds from official test labels.

• Future work could test audio augmentation, more diverse mixture generation, calibrated probabilities, and stronger temporal aggregation, but these are recommendations rather than completed project components.

• Future work may explore MFCC, modified group delay, tempograms, source separation, and explainability.

# 9. Conclusion

The project implements two complementary musical instrument-recognition tasks using Mel-spectrogram features and convolutional neural networks (CNNs). Stage 1 focuses on identifying the predominant instrument in an audio recording and achieves a held-out test accuracy of 75.25%. Stage 2 extends the system to multilabel recognition, enabling the detection of multiple instruments within the same recording. It uses synthetically generated pair and triplet mixtures to improve the representation of different instrument combinations and employs a 306,155-parameter multilabel CNN with sigmoid outputs. The model is evaluated on 2,874 official IRMAS test recordings, with performance assessed using micro-F1 and macro-F1 scores. Class-specific threshold optimization improves the official micro-F1 score from 0.6208 to 0.6302 compared with a global threshold of 0.25. The results demonstrate the potential of combining digital signal processing with deep learning for instrument recognition, while also highlighting the difficulty of detecting overlapping instruments in polyphonic audio. Performance generally decreases as the number of simultaneously present instruments increases, although the highest-polyphony groups contain relatively few recordings, limiting the strength of conclusions for those groups.

# References

[1] P. Sivalingam, A. K. T. J., S. Krishna, Y. Reddy, R. S., and L. C. R., “Robust CNN-based Musical Instrument Recognition with Enhanced Feature Learning,” in Proceedings of the International Conference on Inventive Computation Technologies (ICICT), 2025, pp. 851–857, doi: 10.1109/ICICT64420.2025.11005342.

[2] A. Kratimenos, K. Avramidis, C. Garoufis, A. Zlatintsi, and P. Maragos, “Augmentation Methods on Monophonic Audio for Instrument Classification in Polyphonic Music,” in Proceedings of the European Signal Processing Conference (EUSIPCO), 2020, doi: 10.23919/Eusipco47968.2020.9287745.

**Relevance of Reference [1] — Primary Reference (Base Paper)**

**The first research paper serves as the primary reference (base paper) for this project, and a copy of the paper is attached along with this report for reference.**

The paper *“Robust CNN-based Musical Instrument Recognition with Enhanced Feature Learning”* (2025) presents a CNN-based approach for recognizing 11 musical instrument classes using Mel-spectrogram representations. Its audio preprocessing pipeline, including resampling to 22,050 Hz, STFT-based feature extraction, and the use of 128 Mel bands, a 2,048-point FFT, and a hop length of 512 samples, closely relates to the methodology adopted in our project. The paper also provides useful guidance on CNN architecture, model training, and performance evaluation. Our project builds on this single-label classification approach by extending the system to multilabel recognition of multiple instruments.

**Relevance of Reference [2] — Supporting Research Paper**

The paper *“Augmentation Methods on Monophonic Audio for Instrument Classification in Polyphonic Music”* (2020) focuses on improving instrument recognition by generating artificial polyphonic examples from recordings of individual instruments. It investigates different mixing strategies and demonstrates the potential of augmentation for recognizing multiple instruments in complex musical recordings. This work is particularly relevant to the multilabel stage of our project, which uses synthetic two-instrument mixtures for model training. It also provides insight into the challenges of overlapping instrument sounds and the importance of representative training data when evaluating models on real polyphonic recordings.

**Overall Relevance to the Project**

Together, these papers provide a foundation for the two main aspects of the project: predominant-instrument classification using Mel-spectrograms and CNNs, and multilabel recognition using synthetically generated mixtures. Their methodologies and findings help guide the selection of audio preprocessing techniques, model architectures, training

# Appendix A — Selected Implementation Code

The following selected code excerpts illustrate key parts of the audio-processing and model-training pipeline. Some excerpts are shortened for readability.

A1. Audio loading and sampling-rate conversion

```
y, sr = librosa.load(file_path, sr=22050, mono=True)
```

The loader converts the audio to mono and resamples it to 22,050 Hz.

A2. Mel-spectrogram and decibel conversion

```
mel = librosa.feature.melspectrogram(
  y=segment,
  sr=sr,
  n_fft=2048,
  hop_length=512,
  n_mels=128,
  window="hann"
)
mel_db = librosa.power_to_db(mel, ref=np.max)
mel_normalized = (mel_db + 80.0) / 80.0
```

The audio preprocessing stage converts each recording into a Mel-spectrogram using the short-time Fourier transform (STFT) with a sampling rate of 22,050 Hz, an FFT size of 2,048, and a hop length of 512 samples. The spectrogram is mapped to 128 Mel frequency bands and converted into a decibel scale to represent signal power. Finally, the values are normalized to the range [0, 1], providing a consistent input representation for the CNN model and facilitating effective feature learning during training.

A3. Stage 1 deeper CNN definition

```
model_deep = tf.keras.Sequential([
  tf.keras.layers.Input(shape=(128, 130, 1)),
  tf.keras.layers.Conv2D(32, (3,3), activation='relu', padding='same'),
  tf.keras.layers.BatchNormalization(),
  tf.keras.layers.Conv2D(32, (3,3), activation='relu', padding='same'),
  tf.keras.layers.BatchNormalization(),
  tf.keras.layers.MaxPooling2D((2,2)),
  tf.keras.layers.Dropout(0.25),
  tf.keras.layers.Conv2D(64, (3,3), activation='relu', padding='same'),
  tf.keras.layers.BatchNormalization(),
  tf.keras.layers.Conv2D(64, (3,3), activation='relu', padding='same'),
  tf.keras.layers.BatchNormalization(),
  tf.keras.layers.MaxPooling2D((2,2)),
  tf.keras.layers.Dropout(0.25),
  tf.keras.layers.Conv2D(128, (3,3), activation='relu', padding='same'),
  tf.keras.layers.BatchNormalization(),
  tf.keras.layers.Conv2D(128, (3,3), activation='relu', padding='same'),
  tf.keras.layers.BatchNormalization(),
  tf.keras.layers.MaxPooling2D((2,2)),
  tf.keras.layers.Dropout(0.25),
  tf.keras.layers.GlobalAveragePooling2D(),
  tf.keras.layers.Dense(128, activation='relu'),
  tf.keras.layers.Dropout(0.5),
  tf.keras.layers.Dense(11, activation='softmax')
])
```

This excerpt shows the Stage 1 CNN architecture using global average pooling.

The Stage 1 model uses a deeper CNN architecture to extract meaningful features from Mel-spectrogram inputs of size 128 × 130 × 1. Multiple convolutional layers with ReLU activation learn increasingly complex time-frequency patterns, while batch normalization improves training stability. Max-pooling reduces feature-map dimensions, and dropout helps minimize overfitting. Global average pooling converts the extracted feature maps into a compact representation, followed by dense layers that perform classification into 11 instrument classes using the softmax activation function.

A4. Stage 2 multilabel CNN definition

```
model_deep = tf.keras.Sequential([
  tf.keras.layers.Input(shape=(128, 130, 1)),
  tf.keras.layers.Conv2D(32, (3,3), activation='relu', padding='same'),
  tf.keras.layers.BatchNormalization(),
  tf.keras.layers.Conv2D(32, (3,3), activation='relu', padding='same'),
  tf.keras.layers.BatchNormalization(),
  tf.keras.layers.MaxPooling2D((2,2)),
  tf.keras.layers.Dropout(0.25),
  tf.keras.layers.Conv2D(64, (3,3), activation='relu', padding='same'),
  tf.keras.layers.BatchNormalization(),
  tf.keras.layers.Conv2D(64, (3,3), activation='relu', padding='same'),
  tf.keras.layers.BatchNormalization(),
  tf.keras.layers.MaxPooling2D((2,2)),
  tf.keras.layers.Dropout(0.25),
  tf.keras.layers.Conv2D(128, (3,3), activation='relu', padding='same'),
  tf.keras.layers.BatchNormalization(),
  tf.keras.layers.Conv2D(128, (3,3), activation='relu', padding='same'),
  tf.keras.layers.BatchNormalization(),
  tf.keras.layers.MaxPooling2D((2,2)),
  tf.keras.layers.Dropout(0.25),
  tf.keras.layers.GlobalAveragePooling2D(),
  tf.keras.layers.Dense(128, activation='relu'),
  tf.keras.layers.Dropout(0.5),
  tf.keras.layers.Dense(11, activation='sigmoid')
])
```

This is a compact excerpt: the 64- and 128-filter blocks are represented by a comment; the full architecture is shown in Table 8.

A5. Stage 2 compilation

```
model_multilabel_v2.compile(
  optimizer=tf.keras.optimizers.Adam(),
  loss='binary_crossentropy',
  metrics=[
    tf.keras.metrics.BinaryAccuracy(name='binary_accuracy'),
    tf.keras.metrics.AUC(name='auc', multi_label=True)
  ]
)
```

Binary cross-entropy scores each instrument-presence output independently.

A6. Training callbacks and checkpointing

```
callbacks_v2 = [
  tf.keras.callbacks.EarlyStopping(
    monitor='val_loss', patience=7,
    restore_best_weights=True, verbose=1
  ),
  tf.keras.callbacks.ReduceLROnPlateau(
    monitor='val_loss', factor=0.5, patience=3,
    min_lr=1e-6, verbose=1
  ),
  tf.keras.callbacks.ModelCheckpoint(
    checkpoint_path_v2, monitor='val_loss',
    save_best_only=True, verbose=1
  )
]
```

These callbacks implement early stopping, learning-rate reduction, and best-model checkpointing.

These callbacks improve the efficiency and reliability of model training. Early stopping monitors validation performance and stops training when no further improvement is observed, helping prevent overfitting. Learning-rate reduction decreases the learning rate when validation performance plateaus, allowing the model to converge more effectively. Best-model checkpointing saves the model weights corresponding to the best validation performance, ensuring that the most effective version of the trained model is retained for evaluation.

A7. Synthetic mixture targets

```
X_train_v2 = np.concatenate(
  [X_original_train_ml, X_balanced_train, X_triplet_train],
  axis=0
)
Y_train_v2 = np.concatenate(
  [Y_original_train_ml, Y_balanced_train, Y_triplet_train],
  axis=0
)
```

This excerpt illustrates how original clips and synthetic pair/triplet mixtures are combined into a training set.

A8. Threshold-based classification

```
val_pred = (val_prob_exp3 >= threshold).astype(int)
micro_f1 = f1_score(
  Y_val_exp3, val_pred,
  average='micro', zero_division=0
)
macro_f1 = f1_score(
  Y_val_exp3, val_pred,
  average='macro', zero_division=0
)
```

The implementation evaluates candidate thresholds and selects the one that maximizes validation micro-F1.

A9. Long-audio prediction window setup

```
def predict_long_audio_final(
  file_path, window_seconds=3.0, hop_seconds=1.5
):
  y, sr = librosa.load(file_path, sr=22050, mono=True)
  # Remaining executed body creates 3 s windows at 1.5 s hops,
  # pads the final short window, extracts Mel features,
  # predicts each window, and aggregates window probabilities.
```

The excerpt is intentionally shortened; the comments identify omitted body portions.

| Aspect | Base paper [1] | Group 4 Stage 1 | Group 4 Stage 2 extension |
|---|---|---|---|
| Task | 11-class predominant instrument classification | 11-class predominant instrument classification | Multi-label instrument-presence recognition |
| Input features | 128 Mel bands; 128×128×1 input; 2,048 FFT; 512 hop | 128 Mel bands; 128×130×1 input; 2,048 FFT; 512 hop | Same 128×130×1 representation; dB mapping to [0,1] |
| Output | 11-way softmax | 11-way softmax | 11 independent sigmoid outputs |
| Architecture size | 440,075 parameters (paper Table II) | Implementation versions: 111,499 (GAP) or 4,289,419 (Flatten) | 306,155 parameters (balanced-mixture v2) |
| Training/evaluation | 60 epochs; peak validation accuracy 78.37%; paper conclusion also reports 78.47% test accuracy | Held-out accuracy 75.25%; loss 0.7513; n=1,006 (checkpoint mapping unresolved) | Official IRMAS test: micro-F1 0.6208 global threshold; 0.6302 class-specific |
| Beyond base paper | Single-label baseline | Reproduction/parallel implementation of predominant-class approach | Synthetic pair/triplet training data; multilabel targets; threshold sweeps; long-audio window aggregation; polyphony analysis |

Proceedings of the International Conference on Inventive Computation Technologies (ICICT-2025)
IEEE Xplore Part Number: CFP25F70-ART; ISBN: 979-8-3315-1224-8

# Robust CNN-based Musical Instrument Recognition with Enhanced Feature Learning

Padmesh Sivalingam*, Aamith Kishore T J*, Sri Krishna P*, Yaswanth Reddy B*, Ragav S*, Lekshmi C. R.*
*Amrita School of Artificial Intelligence, Amrita Vishwa Vidyapeetham, Coimbatore, India
Emails: {cb.sc.u4aie24044, cb.sc.u4aie24001, cb.sc.u4aie24054, cb.sc.u4aie24061, cb.sc.u4aie24041}@cb.students.amrita.edu,
cr_lekshmi@cb.amrita.edu

2025 International Conference on Inventive Computation Technologies (ICICT) | 979-8-3315-1224-8/25/$31.00 ©2025 IEEE | DOI: 10.1109/ICICT64420.2025.11005342

***Abstract*—Musical instrument recognition is a challenging task with applications in music information retrieval, audio processing, and automated transcription. This study presents a Convolutional Neural Network (CNN) model leveraging Mel spectrograms from the IRMAS dataset to classify 11 instrument categories. The model, incorporating convolutional layers, batch normalization, and dropout regularization, achieved a peak validation accuracy of 78.37% over 60 epochs. Comparative analysis with state-of-the-art methods highlights its competitive performance and computational efficiency. Robustness evaluations on varying input lengths and noise levels assess the model's generalization. Performance metrics, including accuracy trends, loss curves, and a confusion matrix, demonstrate strong classification for instruments like piano and violin while revealing challenges in distinguishing spectrally similar instruments. These findings underscore the effectiveness of CNNs for instrument classification and provide insights for enhancing deep learning-based audio recognition models.**

***Index Terms*—Musical Instrument Classification, Convolutional Neural Networks (CNN), Mel Spectrograms, Audio Signal Processing, Deep learning**

## I. Introduction

The classification of musical instruments from audio signals is an essential task in music information retrieval (MIR) with applications in automated music transcription, content-based recommendation systems, and digital music analysis. Identifying instruments within an audio recording plays a crucial role in tasks such as music tagging, source separation, and performance analysis, benefiting both academic research and commercial applications.

Despite its significance, instrument classification presents several challenges. The spectral characteristics of different instruments often overlap, making it difficult to distinguish between similar-sounding classes. Variability in recording conditions, background noise, and differences in instrument timbre further complicate the classification process. Additionally, variations in playing techniques introduce intra-class diversity, making it challenging for traditional machine-learning models to achieve high accuracy.

Conventional methods rely on manually extracted audio features, such as spectral centroids and Mel-frequency cepstral coefficients (MFCCs). While these features help characterize audio signals, they often fail to capture intricate patterns necessary for robust classification. Deep learning techniques, particularly Convolutional Neural Networks (CNNs), have emerged as powerful tools for processing audio data. By learning feature representations directly from raw or transformed audio inputs such as spectrograms, CNNs have shown significant success in speech recognition, environmental sound classification, and music genre identification.

This study proposes a CNN-based approach for musical instrument classification using the IRMAS-TrainingData dataset, which contains audio samples from 11 distinct instruments, including guitar, piano, violin, and flute. Our method transforms raw audio signals into Mel spectrograms—a time-frequency representation that effectively captures spectral features—and uses them as input to a CNN model. The architecture incorporates multiple convolutional layers, batch normalization, and dropout techniques to enhance generalization and mitigate overfitting. Additionally, adaptive learning rate adjustments and early stopping mechanisms are implemented to optimize training efficiency.

To assess the effectiveness of our approach, we evaluate its performance using accuracy, loss metrics, and a confusion matrix, providing a comprehensive analysis of its strengths and limitations. Furthermore, we compare our results with state-of-the-art (SOTA) methods and conduct robustness testing by analyzing the model's performance using a confusion matrix and accuracy-loss curves. By leveraging deep learning for instrument classification, this research contributes to the advancement of MIR and audio-based machine learning. The findings highlight the potential of CNNs in musical instrument recognition and offer insights for further improving automated music analysis systems.

## II. Realated Work

Early approaches to musical instrument recognition relied on handcrafted feature extraction combined with classical machine-learning techniques. Heittola et al. [6] employed Non-negative Matrix Factorization (NMF) alongside Mel-frequency cepstral coefficients (MFCCs) to analyze and classify instruments in audio recordings.

Further advancements in feature-based classification refined these early approaches. Kitahara et al. [8] introduced a fusion

*Corresponding author: Lekshmi C. R. (Email: cr_lekshmi@cb.amrita.edu)


Authorized licensed use limited to: Motilal Nehru National Institute of Technology. Downloaded on September 12,2026 at 05:07:21 UTC from IEEE Xplore. Restrictions apply.

model that utilized spectral, temporal, and modulation features with principal component analysis (PCA) to enhance classification accuracy. Fuhrmann et al. [3] extended this research by employing Support Vector Machines (SVMs) to classify instruments based on extracted audio features. Bosch et al. [1] incorporated source separation techniques as a preprocessing step, improving feature extraction and overall recognition performance.

With deep learning advancements, Convolutional Neural Networks (CNNs) significantly improved instrument recognition. Han et al. [5] introduced a Mel-spectrogram-CNN approach with aggregation over sliding windows, which Pons et al. [14] optimized for better timbral feature capture. Gururani et al. [16] applied deep neural networks (DNNs) with temporal max-pooling for instrument detection, while Yu et al. [17] explored multitask learning with auxiliary classification to refine category recognition. Gomez et al. [4] investigated source separation and transfer learning, demonstrating improved performance, particularly for smaller datasets. Soraghan et al. [13] employed the Hilbert-Huang Transform (HHT) with CNNs, while Kratimenos et al. [9] trained VGG-like CNN classifiers on augmented datasets for improved recognition.

Building upon these approaches, Lekshmi et al. [10], [12], [15] explored Mel-spectrogram and phase-based modgdgram representations with data augmentation using WaveGAN. They also experimented with different transformer architectures using an ensemble of Mel-spectrogram, tempogram, and modgdgram representations. Zhang and Bai [18] embedded augmentation-based CNN architectures for better generalization across datasets. Zhong et al. [19] investigated pretraining on isolated musical notes for polyphonic instrument classification using transfer learning. Dash et al. [2] integrated Discrete Wavelet Transform (DWT) with deep CNNs to enhance classification in complex musical arrangements. In contrast, Lekshmi and Rajan [11] explored compact convolutional transformers for multi-instrument recognition, setting new benchmarks in classification accuracy.

Despite advancements in instrument recognition, most studies have focused on multiple predominant instruments rather than solo classification. While CNN-based models have improved performance, challenges persist in generalization, dataset biases, and distinguishing instruments with similar timbres. To address these gaps, our work leverages a CNN with Mel-spectrograms for solo instrument classification using the IRMAS dataset. By incorporating regularization to mitigate overfitting and conducting a detailed performance analysis, we contribute to improving the accuracy and robustness of deep learning-based instrument recognition.

The rest of the paper is organized as follows. Section III represents the system description, Section IV represents the results and analysis. Finally, the paper is concluded in Section V.

## III. System Description

The proposed system classifies musical instruments from the IRMAS-TrainingData dataset using a Convolutional Neural Network (CNN). The process consists of four key stages: data preprocessing, model design, training with evaluation, and performance comparison.

### A. Dataset and Preprocessing

The IRMAS-TrainingData dataset [3] contains 6,705 three-second audio clips in `.wav` format, each corresponding to one of 11 solo instruments: acoustic guitar (gac), clarinet (cla), electric guitar (gel), cello (cel), violin (vio), organ (org), saxophone (sax), trumpet (tru), flute (flu), piano (pia), and voice (voi). Audio files were processed using the Librosa library with a 22,050 Hz sampling rate, ensuring a balance between computational efficiency and audio fidelity.

TABLE I
Summary of the IRMAS-TrainingData Dataset

| Instrument | Number of Samples |
|---|---|
| Acoustic Guitar (gac) | 637 |
| Clarinet (cla) | 505 |
| Electric Guitar (gel) | 760 |
| Cello (cel) | 388 |
| Violin (vio) | 580 |
| Organ (org) | 682 |
| Saxophone (sax) | 615 |
| Trumpet (tru) | 577 |
| Flute (flu) | 595 |
| Piano (pia) | 721 |
| Voice (voi) | 778 |
| **Total** | **6,705** |

### B. Mel Spectrogram Computation

To transform raw audio into a suitable format for the CNN, we computed Mel spectrograms, a time-frequency representation that aligns with human auditory perception. The Mel spectrograms were generated with 128 Mel bands, a hop length of 512 samples, and an FFT window size of 2,048, ensuring a balance between frequency resolution and temporal granularity [5]. The resulting power spectrograms were converted to a decibel scale using Librosa's `power_to_db` function to emphasize amplitude variations, which are critical for distinguishing instrument timbres [5].

To ensure uniformity across all spectrograms, we addressed shape inconsistencies by defining a target shape of $128 \times 128$ pixels. Spectrograms smaller than this size were padded with zeros, while larger ones were truncated using a custom `pad_or_truncate` function. This step was crucial for batch processing in the CNN, which requires fixed-size inputs. The spectrograms were then normalized using min-max scaling to map their values to the range [0, 1], reducing the impact of amplitude variations across samples. Finally, a channel dimension was added to each spectrogram, resulting in an input shape of (128, 128, 1), making the data compatible with the 2D convolutional layers of the CNN.

The Mel spectrogram is computed by first applying the Short-Time Fourier Transform (STFT) to the audio signal $x(t)$, followed by a Mel filterbank transformation. The STFT is defined as:


Authorized licensed use limited to: Motilal Nehru National Institute of Technology. Downloaded on September 12,2026 at 05:07:21 UTC from IEEE Xplore. Restrictions apply.

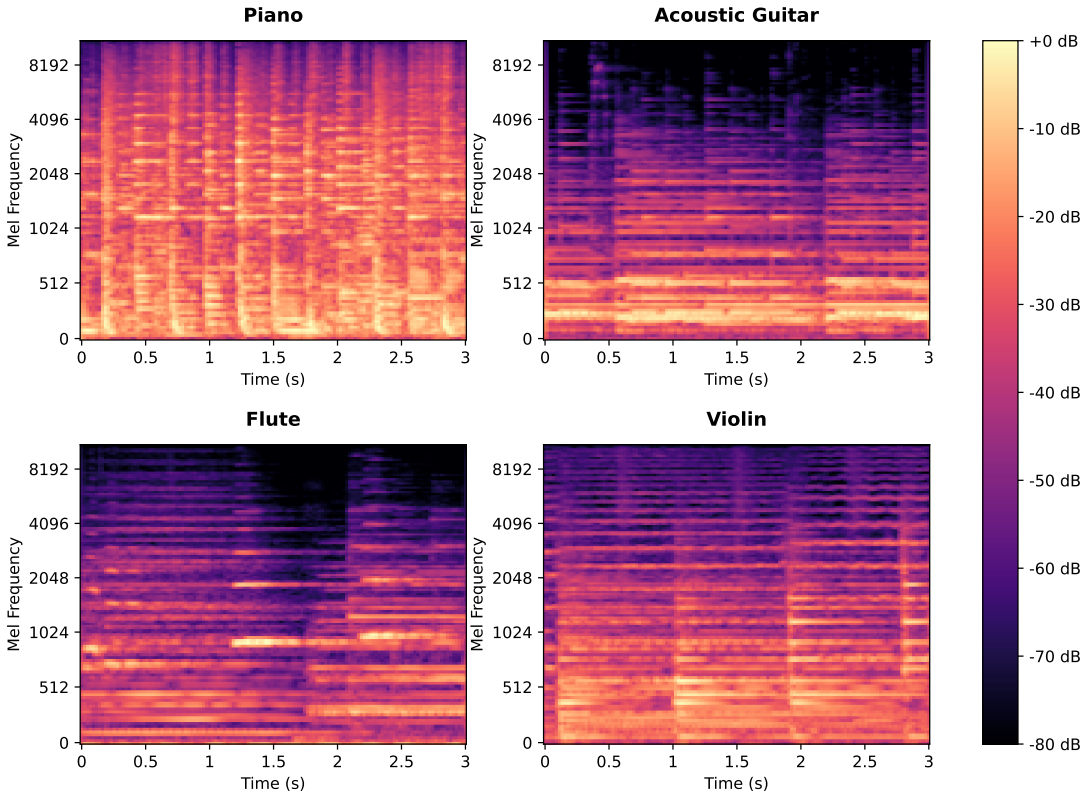



Fig. 1. Mel spectrograms of selected instruments: Piano, Acoustic Guitar, Flute, and Violin. Each spectrogram represents the time-frequency distribution of the corresponding instrument.

$$S(t,f) = \sum_{n=0}^{N-1} x(n)w(n-t)e^{-j2\pi fn/N}, \tag{1}$$

where $w(n)$ is the window function, $N$ is the FFT window size (2,048), and $f$ is the frequency bin.

The Hann window function, used to reduce spectral leakage, is given by:

$$w(n) = 0.5\left(1 - \cos\left(\frac{2\pi n}{N-1}\right)\right). \tag{2}$$

The power spectrogram $|S(t,f)|^2$ is then passed through a Mel filterbank with 128 bands to obtain the Mel spectrogram:

$$S_{\text{Mel}}(t,m) = \sum_f H(m,f)\cdot|S(t,f)|^2, \tag{3}$$

where $H(m,f)$ represents the Mel filter weights.

To convert from frequency (Hz) to the Mel scale, we use:

$$f_{\text{mel}} = 2595\log_{10}\left(1 + \frac{f}{700}\right). \tag{4}$$

The inverse Mel scale transformation, used when necessary, is given by:

$$f = 700\left(10^{\frac{f_{\text{mel}}}{2595}} - 1\right). \tag{5}$$

The computed Mel spectrogram is converted to a decibel scale as:

$$S_{\text{dB}}(t,m) = 10\log_{10}(S_{\text{Mel}}(t,m)). \tag{6}$$

Finally, min-max normalization is applied to ensure consistency across spectrograms:

$$S_{\text{norm}}(t,m) = \frac{S_{\text{dB}}(t,m) - S_{\text{min}}}{S_{\text{max}} - S_{\text{min}}}, \tag{7}$$

where $S_{\text{min}}$ and $S_{\text{max}}$ are the minimum and maximum values of the decibel-scaled spectrogram. Figure 1 shows sample Mel-spectrograms from the dataset.

These steps ensure that the extracted spectrogram features are standardized and well-suited for CNN-based classification.

### C. Model Architecture

The CNN model was implemented using TensorFlow and Keras, designed to extract hierarchical features from Mel spectrograms for instrument classification. The architecture consists of three convolutional blocks, followed by global average pooling and dense layers. The first block includes a Conv2D layer with 64 filters (3×3 kernel), ReLU activation, and L2 regularization (coefficient of 0.0001) to prevent overfitting by penalizing large weights. This is followed by batch normalization to stabilize training by normalizing the activations, and a 2×2 max-pooling layer to reduce spatial


Authorized licensed use limited to: Motilal Nehru National Institute of Technology. Downloaded on September 12,2026 at 05:07:21 UTC from IEEE Xplore. Restrictions apply.

TABLE II
CNN MODEL ARCHITECTURE SUMMARY

| Layer | Type | Output Shape | Params |
|---|---|---|---|
| Input | Input Layer | (128, 128, 1) | 0 |
| Conv2D | 64 filters (3×3), ReLU | (126, 126, 64) | 640 |
| BatchNorm | Batch Norm | (126, 126, 64) | 256 |
| MaxPool | 2×2 Pooling | (63, 63, 64) | 0 |
| Conv2D | 128 filters (3×3), ReLU | (61, 61, 128) | 73,856 |
| BatchNorm | Batch Norm | (61, 61, 128) | 512 |
| MaxPool | 2×2 Pooling | (30, 30, 128) | 0 |
| Conv2D | 256 filters (3×3), ReLU | (28, 28, 256) | 295,168 |
| BatchNorm | Batch Norm | (28, 28, 256) | 1,024 |
| MaxPool | 2×2 Pooling | (14, 14, 256) | 0 |
| GlobalAvgPool | Pooling Layer | (256) | 0 |
| Dense | Fully Connected (256), ReLU | (256) | 65,792 |
| Dropout | Dropout (50%) | (256) | 0 |
| Dense | Fully Connected (11), Softmax | (11) | 2,827 |
| **Total Params** | | | **440,075** |

dimensions while retaining important features. The second and third blocks follow a similar structure but increase the number of filters to 128 and 256, respectively, allowing the model to learn more complex patterns as the depth increases. The progressive increase in filters reflects the need to capture both low-level features (e.g., edges in the spectrogram) and high-level features (e.g., instrument-specific spectral patterns).

After the convolutional blocks, a global average pooling layer reduces the feature maps to a 1D vector, eliminating the need for a flattening operation and reducing the number of parameters, which helps mitigate overfitting. The resulting vector is fed into a dense layer with 256 units and ReLU activation, also with L2 regularization, to learn non-linear combinations of the extracted features. A dropout layer with a 50% rate is applied to randomly disable half of the neurons during training, further preventing overfitting by promoting robustness in the learned features [7]. The final layer is a dense layer with 11 units (one per instrument class) and a softmax activation, producing a probability distribution over the classes. The model was compiled with the Adam optimizer (initial learning rate of 0.0005) for adaptive gradient updates, categorical cross-entropy loss for multi-class classification, and accuracy as the evaluation metric.

The model's architecture and parameter distribution are summarized in Table II.

The softmax activation in the final layer computes the probability distribution over the 11 instrument classes as:

$$p(y_i) = \frac{e^{z_i}}{\sum_{j=1}^{11} e^{z_j}}, \tag{8}$$

where $z_i$ is the input to the softmax layer for class $i$, and $p(y_i)$ is the predicted probability of class $i$.

This architecture effectively balances complexity and generalization, ensuring robust feature extraction while mitigating overfitting through regularization and dropout.

### *D. Training and Evaluation*

The dataset was split into training (80%) and test (20%) sets using stratified sampling to ensure that the class distribution was preserved, addressing the potential for class imbalance in the IRMAS dataset. The labels, originally integers representing the 11 instrument classes, were converted to one-hot encoded format using Keras' `to_categorical` function, enabling the model to perform multi-class classification. The model was trained for 60 epochs with a batch size of 32, a choice that balances computational efficiency with gradient stability. The Adam optimizer was used along with categorical cross-entropy as the loss function to enhance learning efficiency and handle the multi-class classification task effectively.

To improve training dynamics and prevent overfitting, two callbacks were employed. The `ReduceLROnPlateau` callback monitored the validation loss and reduced the learning rate by a factor of 0.5 (down to a minimum of 1e-6) if the loss did not improve for 3 consecutive epochs, allowing the model to make finer adjustments as it approached convergence. The `EarlyStopping` callback halted training if no improvement was observed for 10 epochs and restored the model weights from the epoch with the best validation performance to ensure optimal generalization. Table III summarizes the key hyperparameters used in the training process.

TABLE III
KEY HYPERPARAMETERS FOR TRAINING THE CNN MODEL

| Hyperparameter | Value |
|---|---|
| Initial Learning Rate | 0.0005 |
| Batch Size | 32 |
| Number of Epochs | 60 |
| L2 Regularization Coefficient | 0.0001 |
| Dropout Rate | 50% |
| Learning Rate Reduction Factor | 0.5 |
| Minimum Learning Rate | 1e-6 |
| Early Stopping Patience | 10 epochs |
| ReduceLROnPlateau Patience | 3 epochs |
| Optimizer | Adam |
| Loss Function | Categorical Cross-Entropy |

During training, the model's performance was tracked using accuracy and loss metrics for both the training and validation


Authorized licensed use limited to: Motilal Nehru National Institute of Technology. Downloaded on September 12,2026 at 05:07:21 UTC from IEEE Xplore. Restrictions apply.

sets. For evaluation, a confusion matrix was generated to analyze the model's classification performance across the 11 instrument classes. The matrix was computed by predicting the classes of the test set, converting the softmax outputs to class labels using `argmax`, and comparing them to the true labels. Additionally, accuracy and loss curves were plotted over the 60 epochs to visualize the training dynamics and assess the model's generalization ability. These metrics provide a comprehensive view of the system's strengths and limitations, as discussed in the results section.

### *E. Performance Comparison*

The performance of the proposed model is compared with state-of-the-art methods, including the Han model [5], Pons model [14], and a deep neural network (DNN) trained on handcrafted music texture features. Han [5] and Pons [14] employed deep CNN architectures for musical instrument recognition, incorporating a sliding window analysis and aggregation strategy. In their approaches, multiple sigmoid outputs were aggregated and thresholded to form the final prediction. Both models were evaluated using 1-second audio slices. In contrast, our method does not rely on sliding window analysis or aggregation strategies.

Additionally, we compare the proposed method with a model utilizing handcrafted music features such as spectral centroid, chroma short-time Fourier transform (chroma-STFT), MFCCs (13 coefficients), spectral roll-off, zero-crossing rate, and root mean square energy (RMSE), which were fed into a deep neural network (DNN). This DNN consists of seven layers, with the number of filters increasing from 8 to 512. The Adam optimizer was used with categorical cross-entropy loss for training.

## IV. Results and Analysis

The proposed CNN model was trained and evaluated on the IRMAS dataset, achieving a peak validation accuracy of 78.37% at epoch 51 during the 60-epoch training process. The training logs indicate a steady improvement in both training and validation accuracy, with the learning rate being adaptively reduced from 0.0005 to 1e-6 over the course of training. Below, we present a detailed analysis of the model's performance using the confusion matrix, accuracy trends, and loss curves. Also, we compared our proposed CNN with state-of-the-art architectures.

### *A. Confusion Matrix Analysis*

Figure 3 illustrates the confusion matrix for the test set, where the rows represent true instrument labels, and the columns represent predicted labels for the 11 instrument classes (gac: guitar acoustic, cla: clarinet, gel: guitar electric, cel: cello, vio: violin, org: organ, sax: saxophone, tru: trumpet, flu: flute, pia: piano, voi: voice).

The diagonal values indicate correct classifications, with high accuracy for piano (133), violin (142), and guitar acoustic (115). This suggests that the model performs well in recognizing these instruments. However, some misclassifications occur due to timbral similarities:

- Clarinet (cla) and Flute (flu): 10 clarinet samples were misclassified as flute, possibly due to overlapping spectral characteristics. - Guitar Electric (gel) and Guitar Acoustic (gac): 9 samples of guitar electric were classified as guitar acoustic, likely due to similar harmonic structures. - Saxophone (sax) and Trumpet (tru): There are a few cases of saxophone being confused with the trumpet, likely because both belong to the brass family and share similar frequency ranges.

These results indicate that while the CNN model captures distinguishing features for most instruments, fine-grained spectral distinctions remain challenging. Additional training data, augmentation techniques, or contrastive learning approaches could help mitigate these misclassifications.

### *B. Model Accuracy and Loss Trends*

Figure 2 presents the training and validation accuracy and loss over 60 epochs. The training accuracy steadily increases, reaching 90.96% by the final epoch. The validation accuracy peaks at 78.37% at epoch 51, before slightly declining to 78.00%, indicating a potential overfitting issue despite regularization.

The training loss decreases consistently to 0.39, suggesting effective learning. The validation loss stabilizes at 0.73 after an initial spike, indicating that while the model generalizes reasonably well, further optimization could improve performance. The gap between training and validation accuracy suggests overfitting, which could be addressed by using data augmentation techniques such as pitch shifting, time stretching, and noise injection. Adding more dropout layers or stronger L2 regularization may also help reduce overfitting.

Additionally, the confusion matrix highlights misclassification issues for spectrally similar instruments. Potential solutions include incorporating self-supervised pretraining on larger unlabeled instrument datasets and using attention mechanisms to focus on discriminative spectral features. From a computational standpoint, the CNN model achieved reasonably fast convergence. Further tuning with a learning rate scheduler and adaptive batch sizes could enhance training efficiency and reduce unnecessary parameter updates.

### *C. Comparison with State-of-the-Art*

To assess the performance of the proposed CNN model, we compare it with state-of-the-art (SOTA) methods, including the Han model [5], the Pons model [14], and a deep neural network (DNN) trained on handcrafted features such as MFCCs and music texture features (MTF). Table IV summarizes the comparison results, demonstrating that our approach achieves the highest accuracy while maintaining a significantly lower parameter count compared to the Han and Pons models.

The results indicate that the proposed CNN model achieves the highest accuracy while maintaining a significantly lower parameter count compared to other models. Despite having fewer parameters than the Han and Pons models, our approach surpasses them in classification performance, highlighting its efficiency. Also, we are not using sliding window analysis


Authorized licensed use limited to: Motilal Nehru National Institute of Technology. Downloaded on September 12,2026 at 05:07:21 UTC from IEEE Xplore. Restrictions apply.

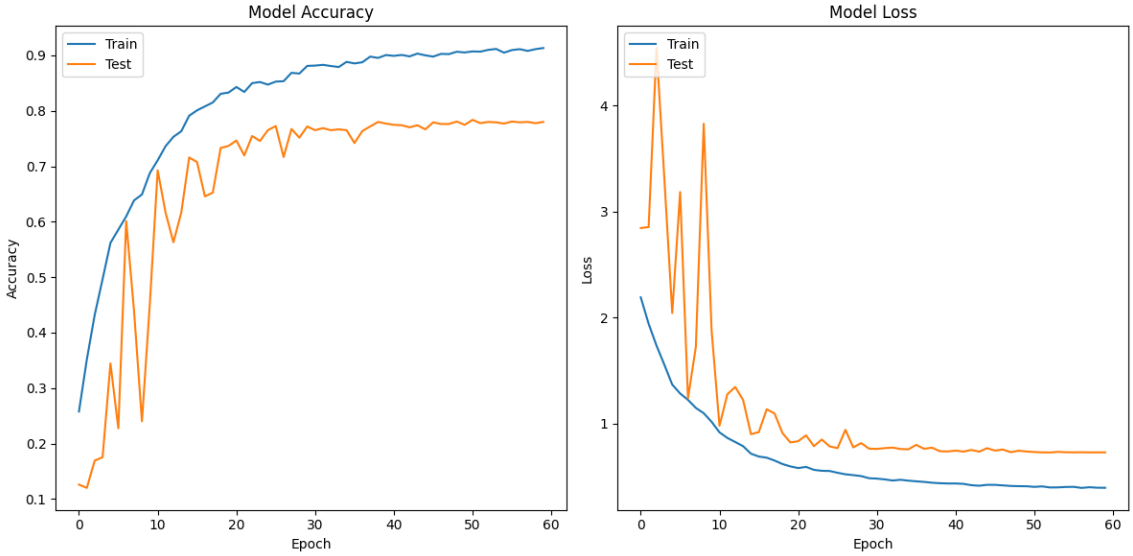



Fig. 2. Training and validation accuracy and loss over 60 epochs.

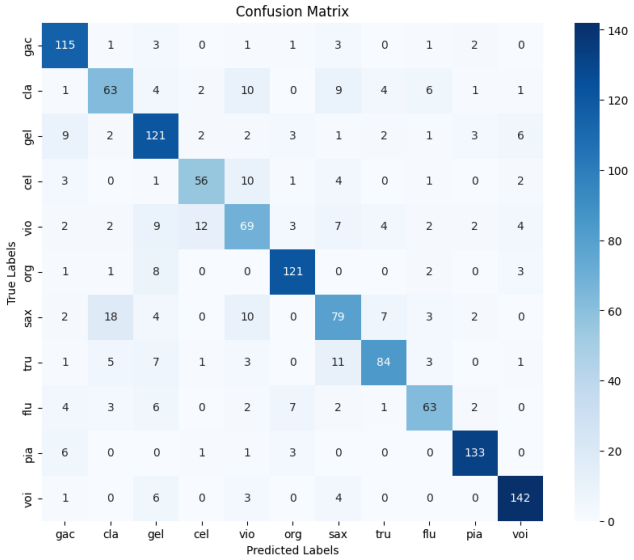



Fig. 3. Confusion Matrix for the test set, showing the classification performance across the 11 instrument classes.

TABLE IV
COMPARISON OF THE PROPOSED METHOD WITH STATE-OF-THE-ART MODELS

| Method | Model Parameters | Test Accuracy (%) |
|---|---|---|
| Han et al. [5] | 1442k | 76.5 |
| Pons et al. [14] | 743k | 75.2 |
| MTF-DNN | NA | 70.8 |
| **Proposed CNN Model** | **440k** | **78.37** |

and aggregation strategy for our method. The MTF + DNN approach, which relies on handcrafted features, achieves the lowest accuracy, suggesting that deep feature extraction from Mel spectrograms is more effective for instrument classification. These findings reinforce the advantage of using a well-optimized CNN architecture for musical instrument recognition.

## V. CONCLUSION

This study proposed a CNN-based approach for musical instrument classification using Mel spectrograms, achieving a test accuracy of 78.47% on the IRMAS dataset. The model demonstrated strong performance in learning discriminative features and accurately classifying predominant instruments, even under challenges such as background noise and overlapping spectral content. Error analysis indicated that confusion often occurred between instruments with similar timbral qualities, such as clarinet and flute, highlighting the inherent difficulty in distinguishing closely related classes. While the model shows promising results, limitations include its dependency on labeled data and reduced accuracy for spectrally similar instruments. Overall, the findings underscore the effectiveness of CNNs for instrument recognition tasks and contribute to advancements in music information retrieval, with potential applications in automatic transcription, music search, and genre classification. As a future direction, the exploration of attention-based models and alternative time-frequency representations may further improve classification performance.

## REFERENCES

[1] J. J. Bosch, J. Janer, F. Fuhrmann, and P. Herrera. Acomparison of sound segregation techniques for predominant instrument recognition in musical audio signals. *in Proc. of 13th Int. Society for Music Information Retrieval Conference (ISMIR)*, 2012.

[2] Sukanta Kumar Dash, SS Solanki, and Soubhik Chakraborty. Deep convolutional neural networks for predominant instrument recognition in polyphonic music using discrete wavelet transform. *Circuits, Systems, and Signal Processing*, 43(7):4239–4271, 2024.

[3] F. Fuhrmann and P. Herrera. Polyphonic instrument recognition for exploring semantic similarities in music. *in Proc. of 13th Int. Conf. on Digital Audio Effects DAFx10, Graz, Austria*, 14(1):1–8, 2010.


Authorized licensed use limited to: Motilal Nehru National Institute of Technology. Downloaded on September 12,2026 at 05:07:21 UTC from IEEE Xplore. Restrictions apply.

[4] Juan S Gómez, Jakob Abeßer, and Estefanía Cano. Jazz solo instrument classification with convolutional neural networks, source separation, and transfer learning. In *ISMIR*, pages 577–584, 2018.

[5] Y. Han, J. Kim, and K. Lee. Deep convolutional neural networks for predominant instrument recognition in polyphonic music. *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 25(1):208–221, 2017.

[6] T. Heittola, A. Klapuri, and T. Virtanen. Musical instrument recognition in polyphonic audio using source-filter model for sound separation. *in Proc. of International Society of Music Information Retrieval Conference*, pages 327–332, 2009.

[7] Deepika Jaswal, Sowmya Vishvanathan, and Soman Kp. Image classification using convolutional neural networks. *International Journal of Scientific and Engineering Research*, 5:1661–1668, 06 2014.

[8] T Kitahara, M. Goto, K. Komatani, T. Ogata, and H. G. Okuno. Instrument identification in polyphonic music: Feature weighting to minimize influence of sound overlaps. *EURASIP Journal of Applied Signal Processing*, pages 155–175, 2007.

[9] A. Kratimenos, K Avramidis, C. Garoufis, Athanasia Zlatintsi, and Petros M. Augmentation methods on monophonic audio for instrument classification in polyphonic music. *in Proc.of 28th European Signal Processing Conference (EUSIPCO)*, pages 156–160, 2021.

[10] CR Lekshmi and Rajeev Rajan. Predominant instrument recognition in polyphonic music using convolutional recurrent neural networks. In *International Symposium on Computer Music Multidisciplinary Research*, pages 214–227. Springer, 2021.

[11] CR Lekshmi and Rajeev Rajan. Compact convolutional transformers for multiple predominant instrument recognition in polyphonic music. In *2024 9th International Conference on Communication and Electronics Systems (ICCES)*, pages 01–06. IEEE, 2024.

[12] CR Lekshmi and Rajan Rajeev. Multiple predominant instruments recognition in polyphonic music using spectro/modgd-gram fusion. *Circuits, Systems, and Signal Processing*, 42(6):3464–3484, 2023.

[13] X. Li, K. Wang, J. Soraghan, and J. Ren. Fusion of hilbert-huang transform and deep convolutional neural network for predominant musical instruments recognition. *in Proc. of 9th Int. Conf. on Artificial Intelligence in Music, Sound, Art and Design*, 2020.

[14] J. Pons, O. Slizovskaia, R. Gong, Emilia Gómez, and X. Serra. Timbre analysis of music audio signals with convolutional neural networks. *in Proc. of 2017 25th European Signal Processing Conference (EUSIPCO)*, pages 2744–2748, 2017.

[15] Lekshmi Chandrika Reghunath and Rajeev Rajan. Transformer-based ensemble method for multiple predominant instruments recognition in polyphonic music. *EURASIP Journal on Audio, Speech, and Music Processing*, 2022(1):11, 2022.

[16] G. Siddharth, C. Summers, and A. Lerch. Instrument activity detection in polyphonic music using deep neural networks. *in Proc. of Int. Society for Music Information Retrieval Conference (ISMIR)*, 2018.

[17] Dongyan Yu, Huiping Duan, Jun Fang, and Bing Zeng. Predominant instrument recognition based on deep neural network with auxiliary classification. *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 28:852–861, 2020.

[18] Jian Zhang and Na Bai. Augmentation embedded deep convolutional neural network for predominant instrument recognition. *Applied Sciences*, 13(18):10189, 2023.

[19] Lifan Zhong, Erica Cooper, Junichi Yamagishi, and Nobuaki Minematsu. Exploring isolated musical notes as pre-training data for predominant instrument recognition in polyphonic music. In *2023 Asia Pacific Signal and Information Processing Association Annual Summit and Conference (APSIPA ASC)*, pages 2312–2319. IEEE, 2023.


Authorized licensed use limited to: Motilal Nehru National Institute of Technology. Downloaded on September 12,2026 at 05:07:21 UTC from IEEE Xplore. Restrictions apply.